Learning-Augmented Smooth Integer Programs with PAC-Learnable Oracles

Hao-Yuan He Jiang-Tian Xue Ming Li Zhi-Hua Zhou

National Key Laboratory for Novel Software Technology
School of Artificial Intelligence, Nanjing University
{hehy, xuejt, lim, zhouzh}@lamda.nju.edu.cn

Abstract

We study an *end-to-end* learning-augmented framework for *smooth integer programs*, a broad class of Boolean polynomial optimization problems including MAX-CUT and MAX-k-SAT. An oracle learned from sampled instances predicts a full Boolean assignment for each unseen instance. For fixed-degree objectives, our polynomial-time pipeline, Leap, linearizes the objective around the predicted assignment, solves an error-budgeted linear program, and rounds the fractional solution. We establish two guarantees. First, for any prediction, the approximation loss of Leap degrades gracefully with prediction error: it is controlled by an *error footprint* that quantifies how much mispredicted coordinates can perturb the objective's local coefficients. Second, we bound the pseudo-dimension of the Leap-induced cost class, yielding a PAC-style guarantee for learning an oracle from a predictor class via empirical risk minimization on instances from an unknown distribution. Together, these results give an end-to-end bound that separates the expected footprint loss of the best oracle in the class from the statistical error of learning. In experiments on MAX-CUT and MAX-SAT, Leap substantially repairs imperfect predictions, empirical risk minimization selects effective oracle candidates, and the pipeline scales to application benchmarks with up to two million clauses.

Keywords: smooth integer programs, learning-augmented algorithms, combinatorial optimization, approximation algorithms, PAC learnability

1 Introduction

Combinatorial optimization is a central task in computer science, operations research, and many scientific applications. Canonical problems such as MAX-CUT and MAX-k-SAT ask for the maximization of an objective over a discrete feasible set, and are typically NP-hard; many are also APX-hard (Papadimitriou and Yannakakis, 1991), ruling out a polynomial-time approximation scheme unless P = NP (Arora et al., 1998). In many application settings, however, optimization instances are not encountered in isolation: similar instances are solved repeatedly, creating an opportunity to learn predictions from past data and use them to guide future optimization. This is the premise of *learning-augmented algorithms*, which use data-driven predictions while retaining provable performance guarantees (Lykouris and Vassilvitskii, 2021; Mitzenmacher and Vassilvitskii, 2022).

In this work, we focus on *smooth integer programs*: the maximization of an n-variate degree-d smooth polynomial $p(\mathbf{x})$ over Boolean assignments,

$$\begin{aligned} \max_{\mathbf{x}} \quad & p(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{x} \in \{0,1\}^n. \end{aligned} \tag{d-IP}$$

Informally, smoothness bounds the coefficient of each degree-ℓ monomial by $O(n^{d-\ell})$.[1] This model was introduced by Arora et al. (1999) and is broad enough to capture MAX-SNP problems (Papadimitriou and Yannakakis, 1991), including canonical graph partitioning tasks such as MAX-CUT and Boolean constraint satisfaction problems such as MAX-k-SAT.

Arora et al. (1999) developed PTASs for *dense* smooth integer programs, where $\mathrm{OPT} = \Omega(n^d)$, using exhaustive sampling, linear programming, and rounding. A key insight in their analysis is that local coefficients estimated from a partial assignment can turn the nonlinear objective into a linear one. Because an optimal assignment is unknown, their algorithm samples a small set of variables and enumerates all Boolean assignments on the sample, which takes $n^{O(1/\eta^2)}$ time for a target precision η.

More recently, Bampis et al. (2024) incorporated partial predictions into this sampling-and-LP paradigm through a *parsimonious* oracle. Their oracle predicts labels only on a sampled set of variables, which replaces the enumeration step, and the guarantee degrades with the number of wrong sampled predictions. Like that of Arora et al. (1999), their guarantee applies to *dense* objectives. This result treats the oracle as an external source of advice, leaving open how the oracle itself is obtained.

Because both methods linearize the objective from a random sample of variables, their guarantees control the resulting approximation error through global density, and the bound of Bampis et al. (2024) depends only on the number of mispredicted labels in the sample. Such bounds treat all prediction errors alike, regardless of where they occur. In combinatorial problems, however, prediction errors are not equally consequential. In MAX-CUT on a star graph, for instance, flipping the center's label in an optimal assignment loses every cut edge, whereas flipping a leaf's label loses only one. More generally, two predictions with the same number of incorrect variables can distort the linearized objective to very different degrees, depending on the structure of the instance. Capturing this difference requires an analysis that tracks how individual prediction errors perturb the linearized objective, rather than merely counting how many errors are made.

To capture this structural sensitivity, we let the oracle predict a full Boolean assignment. This oracle model aligns naturally with modern learning pipelines for graphs and formulas, which typically output a label for every variable (Selsam et al., 2019; Pan et al., 2025). A full prediction determines the linearized objective at once, removing both the sampling step of the methods above and the enumeration step of Arora et al. (1999). Without sampling noise, the linearization error arises solely from mispredicted variables. By decomposing the polynomial objective recursively, we can trace how these mispredictions perturb the local coefficients of the linear surrogate.

This leads to an *error footprint* that measures their cumulative influence, yielding approximation guarantees that reflect where the errors occur and extend beyond the dense regime. We further study how to learn such an oracle from sampled instances, establishing an end-to-end learning-augmented framework. This end-to-end viewpoint leads to two natural questions that organize the paper:

1. *Given a predicted assignment that may contain errors, can we turn it into an approximation algorithm whose guarantee degrades gracefully with the prediction error?*

2. *Can one learn such a predictor from sampled instances, so that the resulting algorithm has a PAC-style performance guarantee?*

1.1 Framework and Main Results

We answer these questions through a learning-augmented approximation framework, using MAX-k-SAT as a concrete running example. Consider a MAX-k-SAT formula with clause set $\mathcal{C}$ without

[1]The degree d is fixed; hidden constants may depend on fixed parameters, including density constants.

duplicate clauses, where each clause contains exactly k literals on distinct variables. For a clause C, let C^+ and C^- denote its positive and negative literal indices. The number of satisfied clauses is the Boolean polynomial

$$p(\mathbf{x}) = \sum_{C \in \mathcal{C}} \left(1 - \prod_{i \in C^-} x_i \prod_{i \in C^+} (1 - x_i) \right).$$

The product term is one exactly when clause C is falsified. The objective is a degree-k smooth integer program: each degree-ℓ coefficient is $O(n^{k-\ell})$.

For Q1, we develop LEAP, short for LEarning-augmented Approximation via linear Programming. Given an instance π, an oracle f predicts a full Boolean assignment $f(\pi) = \widehat{\mathbf{x}} \in \{0,1\}^n$. LEAP uses this prediction as a reference point and proceeds in three steps.

(1) **Linearize.** Decompose the objective recursively into variables multiplied by lower-degree local coefficients, and evaluate these coefficients at $\widehat{\mathbf{x}}$ to obtain a linear surrogate. For example, a clause $x_1 \vee x_2$ contributes $x_1 + x_2 - x_1 x_2 = x_1(1 - x_2) + x_2$ to p. Evaluating the local coefficient $1 - x_2$ at $\widehat{\mathbf{x}}$ gives the linear surrogate $x_1(1 - \widehat{x}_2) + x_2$. Mispredicting $\widehat{x}_2$ changes this coefficient by one.

(2) **Relax and round.** For an error budget b, optimize the surrogate by a linear program that allows each local coefficient to vary by as much as b mispredicted coordinates in $\widehat{\mathbf{x}}$ could perturb it, and round the fractional solution to an integral assignment. Since p is multilinear, rounding the coordinates one at a time, each to the better of 0 and 1, does not decrease p; we call this *deterministic rounding*.

(3) **Select.** Since the prediction error is unknown, repeat step (2) over error budgets and return the best rounded assignment.

Given an oracle prediction $\widehat{\mathbf{x}}$, let $\mathbf{z}$ be the assignment returned by LEAP. Its loss relative to OPT is controlled by the error footprint of the prediction. Theorem 3.1 proves this for every multilinear objective; we state it here for MAX-k-SAT.

Theorem 1.1 (Informal version of Theorem 3.1)**.** *Consider a MAX-k-SAT instance with objective p, and let $\mathbf{x}^*$ be an optimal assignment. Let $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$ be the number of coordinates in which the oracle prediction $\widehat{\mathbf{x}}$ differs from $\mathbf{x}^*$.* LEAP *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \geq OPT - 2\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Here $\Phi_\varepsilon(\widehat{\mathbf{x}})$ is the error footprint: it measures how much up to ε prediction errors around $\widehat{\mathbf{x}}$ can change the local coefficients of the decomposed objective.

With a perfect prediction, $\Phi_0(\widehat{\mathbf{x}}) = 0$, and LEAP returns an optimal assignment. LP duality can further sharpen this bound by an instance-dependent factor (Section 3.5). We bound the error footprint in two ways.

A global bound, which uses only the smoothness scale of the objective, gives a *near-dense* statement. LEAP returns an assignment satisfying

$$p(\mathbf{z}) \geq \mathrm{OPT} - O(n^{k-1}\varepsilon).$$

If $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$ for a constant $\gamma \in [0,1]$, the approximation ratio is $1 - O(\varepsilon/n^\gamma)$.

A local bound, which uses instance structure, gives a *bounded-occurrence* statement by charging each prediction error to only $O(\Delta)$ affected clauses. If each literal occurs in at most Δ clauses, then LEAP returns an assignment satisfying

$$p(\mathbf{z}) \geq \mathrm{OPT} - O(\Delta\varepsilon).$$

Since $\mathrm{OPT} = \Omega(n)$ once variables that appear in no clause are removed, the approximation ratio is $1 - O(\Delta\varepsilon/n)$. Crucially, because the relaxation sees prediction errors only through the local coefficients, an error that heavily degrades the objective need not perturb the coefficients if the decomposition order is chosen well. Optimizing the variable order of the decomposition tightens this bound further (Proposition 3.2) and can even eliminate the impact of high-degree errors entirely, as on star graphs (Example 3).

Both statements extend beyond the dense regime $\mathrm{OPT} = \Omega(n^k)$ studied by Arora et al. (1999) and Bampis et al. (2024) to instances with $\mathrm{OPT} = o(n^k)$. For the near-dense regime, Fotakis et al. (2016) give a sampling-based approximation scheme that runs in subexponential time; given a prediction, LEAP runs in polynomial time, with a guarantee that depends on the prediction error. The order of the local bound cannot be improved uniformly: unless P = NP, no polynomial-time algorithm guarantees loss $o(\Delta\varepsilon)$, even when ε is known (Section 4.3).

We next turn from analyzing a given oracle prediction to selecting an oracle from sampled instances. For Q2, we establish a PAC-style guarantee for oracle selection using the algorithm-selection framework of Gupta and Roughgarden (2017). Instances are drawn from an unknown distribution, and each predictor f induces a LEAP algorithm. We choose a predictor by empirical risk minimization (ERM) over the cost of the solutions returned by these induced algorithms, and bound the pseudo-dimension of the resulting cost class. Let $\mathcal{P}(f)$ denote the expected objective value of the solution that LEAP returns with predictor f, and let H upper-bound objective values. For MAX-k-SAT, $\mathcal{P}(f)$ is the expected number of satisfied clauses, and H may be taken as an upper bound on the number of clauses.

The following end-to-end bound combines an ERM guarantee for the induced cost class with the footprint theorem.

Theorem 1.2 (Informal version of Theorem 5.1)**.** *Fix $\alpha > 0$ and $\delta \in (0, 1]$. Let $\mathcal{F}$ be a predictor class, and let $d_{\mathcal{F}}$ be the sum of the VC dimensions of the n classes formed by its coordinate functions. Suppose ERM selects $\hat{f}_{\mathrm{ERM}} \in \mathcal{F}$ from m sampled instances, where*

$$m = \Omega\left(\left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{F}} \log(e d_{\mathcal{F}}) + \log\frac{1}{\delta}\right)\right).$$

Then, with probability at least $1 - \delta$,

$$\mathcal{P}(\hat{f}_{\mathrm{ERM}}) \geq \mathcal{P}^* - \Lambda^{\Phi}_{\mathcal{F}} - 2\alpha.$$

Here $\mathcal{P}^*$ denotes the expected optimum, and $\Lambda^{\Phi}_{\mathcal{F}}$ is the footprint loss of the best predictor in $\mathcal{F}$, defined formally in Section 5.

The bound separates this footprint loss from the statistical error 2α of ERM. For MAX-k-SAT, the two footprint bounds above give $\Lambda^{\Phi}_{\mathcal{F}} = O(n^{k-1}\varepsilon_{\mathcal{F}})$ under the global smoothness bound and $\Lambda^{\Phi}_{\mathcal{F}} = O(\Delta\varepsilon_{\mathcal{F}})$ under bounded occurrence, where $\varepsilon_{\mathcal{F}}$ is the best expected prediction error achieved by a predictor in $\mathcal{F}$.

1.2 Our Contributions

Our main contributions are as follows.

1. *A learning-augmented approximation pipeline for smooth integer programs.* We introduce Leap, which turns a predicted assignment into a linear surrogate of a smooth polynomial objective, optimizes it by linear programming, and rounds the result, using one oracle query and polynomial time (Section 3).

2. *Error-footprint guarantees and their optimal order.* We show that the loss of Leap is at most twice the error footprint of the prediction, and that LP duality can sharpen this bound. The footprint admits a global bound from smoothness and a local bound from instance structure, and both extend beyond the dense regime (Sections 3 and 4). Accounting for the structure of the instance more finely, through the order of the decomposition, tightens the local bound further. Unless P = NP, no polynomial-time algorithm improves the order of this local bound (Section 4.3).

3. *PAC-style guarantees for oracle selection.* We bound the pseudo-dimension of the cost class induced by Leap, which gives a sample-complexity guarantee for ERM-based oracle selection and an end-to-end bound that separates the footprint loss of the best oracle from the statistical error (Section 5).

4. *Empirical evaluation.* We evaluate Leap on MAX-CUT and MAX-SAT instances. The experiments show that Leap substantially reduces the objective gap of imperfect predictions and that selection by empirical induced cost identifies effective oracle candidates. We also compare Leap with state-of-the-art solvers on large MAX-SAT application benchmarks (Section 6).

1.3 Roadmap

We discuss related work in Section 2. Section 3 presents Leap and its error-footprint analysis. Section 4 applies the framework to MAX-CUT and MAX-k-SAT and shows that the resulting local bounds have optimal order. Section 5 establishes the PAC-style guarantee for oracle selection. Section 6 evaluates Leap on MAX-CUT and MAX-SAT instances. Section 7 concludes the paper. The appendix contains supporting material, including extended related work, additional experimental details, omitted proofs, and the smoothness verification for MAX-k-CSP, to which MAX-SNP problems reduce.

2 Related Work

We position the paper against three lines of work: smooth integer programming and dense CSP approximation, learning-augmented approximation algorithms, and PAC-style learning of algorithmic decisions.

Smooth Integer Programs and Prediction-Guided Relaxations. Smooth integer programs were introduced by Arora et al. (1999) as a polynomial framework for dense constraint satisfaction and related MAX-SNP problems. Fotakis et al. (2016) extended this sampling approach to the near-dense scale $\Omega(n^{k-1+\gamma})$, called almost-sparse in their work. More recently, Bampis et al. (2024) incorporated predictions into the sampling-and-LP framework through a parsimonious oracle.

Following the same intuition, our framework uses the oracle prediction as a reference point for constructing the local quantities in the relaxation. The main technical question is how prediction errors affect the relaxation and the rounded solution. Our analysis answers this question by charging prediction errors to the components of the decomposed objective that they can influence, yielding both global smoothness consequences and local consequences when the instance structure permits them; the local consequences have optimal order among polynomial-time algorithms.

Learning-Augmented Approximation Algorithms. For approximation problems, prior work on learning-augmented algorithms has studied several advice models. In label-advice models, the algorithm receives labels that are correct with probability slightly above $1/2$, often under independence or pairwise-independence assumptions (Cohen-Addad et al., 2024; Ghoshal et al., 2025). In subset-advice models, the optimal labels of a random subset of variables are revealed (Cohen-Addad et al., 2024; Attias et al., 2025). In edge-advice models for graph problems, each edge (i, j) provides local predictions about x_i^* and x_j^* (Aamand et al., 2025). These models yield prediction-sensitive guarantees for MAX-CUT, MAX-SAT, and related CSPs. Our framework studies smooth integer programs with a full oracle prediction $\widehat{\mathbf{x}}$. The guarantee does not impose independence assumptions on oracle errors; its loss is measured through $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$ and the error footprint. A more detailed comparison is given in Section A.1.

Learning Algorithmic Decisions from Data. A complementary line of work studies how to learn algorithmic decisions from instance distributions. Gupta and Roughgarden (2017) introduced a PAC framework for algorithm selection, where empirical performance over sampled instances is used to choose from a family of algorithms. The central statistical requirement is a bounded pseudo-dimension for the induced real-valued cost class, which gives uniform convergence from empirical cost to expected cost. This perspective has been developed further for branch-and-bound, algorithm configuration, and related families of algorithmic decisions (Balcan et al., 2018, 2017, 2021; Khodak et al., 2022; Bartlett et al., 2022; Balcan et al., 2024). We use this perspective to study oracle selection. Each predictor f induces a Leap algorithm, and ERM selects a predictor by the cost of the induced algorithm. Combining the pseudo-dimension bound with the error-footprint guarantee gives an end-to-end learnability guarantee for learning-augmented smooth integer programs.

3 Approach

This section develops the formal construction of Leap and proves its approximation guarantee, addressing Q1. We first recall smooth Boolean polynomial objectives and their recursive decompositions. We then study quadratic objectives as a motivating special case, where the surrogate objective and the error footprint have their simplest form. We use this case to formulate the general footprint relaxation for fixed-degree smooth objectives, prove its approximation guarantee, and sharpen the guarantee through LP duality. We close by deriving global and local bounds on the error footprint, and show how a suitable decomposition order improves the guarantee. The main notation is summarized in Table 1.

3.1 Preliminaries

We recall the smooth polynomial model and the recursive decomposition used to construct the relaxation.

Symbol	Meaning
n, d	Number of Boolean variables and polynomial degree.
p, β	Smooth polynomial objective and its smoothness parameter.
$\mathbf{x}^*, \mathrm{OPT}$	An optimal assignment and the optimal value $p(\mathbf{x}^*)$.
$\widehat{\mathbf{x}}$	Oracle prediction.
ε	Prediction error $\|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$.
b	Error budget of the footprint relaxation.
$\mathbf{y}_b, \mathbf{z}_b$	LP solution and its rounding at budget b; $\mathbf{z}$ is the final output.
$\Phi_b(\widehat{\mathbf{x}})$	Error footprint at prediction $\widehat{\mathbf{x}}$ and budget b.
σ, L_j^σ	Variable order of the decomposition and ordered load of coordinate j.
dgn	Degeneracy of a graph or formula.
Π, π	Instance space and an instance.
$\mathcal{F}, f$	Predictor class and a predictor $f : \Pi \to \{0,1\}^n$.

Table 1: Main notation.

Definition 3.1 (β-smooth)**.** An n-variate polynomial $p(\mathbf{x})$ of degree at most d is *β-smooth* if there exists a constant $\beta > 0$ such that for every $0 \le \ell \le d$, the absolute value of the coefficient of any degree-ℓ monomial in the expansion of $p(\mathbf{x})$ is bounded by $\beta n^{d-\ell}$.

Many fundamental optimization problems have smooth polynomial objective formulations. We recall two examples used throughout the paper.

Example 1. Given an undirected graph $G = (V, E)$, MAX-CUT partitions V to maximize the number of crossing edges. We define the objective over $\mathbf{x} \in \{0,1\}^n$, with $x_i = 1$ denoting that vertex i is in the subset, as

$$p(\mathbf{x}) = \sum_{\{i,j\} \in E} \big(x_i(1 - x_j) + x_j(1 - x_i)\big),$$

which is 2-smooth.

Example 2. Given a formula in conjunctive normal form without duplicate clauses, where each clause has exactly k literals on distinct variables, MAX-k-SAT maximizes the number of satisfied clauses. The objective function is

$$p(\mathbf{x}) = \sum_{C \in \mathcal{C}} \left(1 - \prod_{i \in C^-} x_i \prod_{i \in C^+} (1 - x_i)\right),$$

where C^+ and C^- denote indices of positive and negative literals in clause C. This degree-k polynomial is 4^k-smooth.

Further smoothness verification for MAX-k-CSP is given in Section E.

Multilinear reduction. Since $x_i^2 = x_i$ on $\{0,1\}^n$, every smooth objective can be reduced to multilinear form before the relaxation is built. Replacing every power x_i^e with $e \ge 1$ by x_i and collecting like monomials turns a degree-d, β-smooth polynomial p into a multilinear polynomial $\tilde{p}$ of degree at most d that agrees with p on $\{0,1\}^n$. A degree-ℓ monomial of $\tilde{p}$, for $\ell \ge 1$, collects at most $\binom{d}{\ell} \le 2^d$ monomials of p, each of degree at least ℓ and hence with coefficient at most $\beta n^{d-\ell}$, so $\tilde{p}$ is $2^d\beta$-smooth with respect to the degree bound d. The reduction preserves all Boolean objective values and optimal assignments, so we work from now on with the reduced multilinear objective, still denoted by p, and compute all footprint quantities from it.

The main algebraic tool is a recursive decomposition of p. The root is indexed by $\emptyset$, with $p_\emptyset = p$. For a node I, its children are indexed by (I, j), $j \in [n]$.

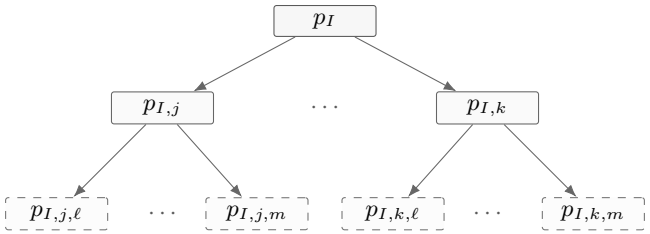


Figure 1: A schematic view of the recursive decomposition. Each node represents a component p_I.

Lemma 3.1. *Let $p(\mathbf{x})$ be an n-variate β-smooth polynomial of degree d. For every node I whose component p_I has positive degree, there exist a constant c_I and child components $p_{I,j}$ such that*

$$p_I(\mathbf{x}) = c_I + \sum_{j\in[n]} x_j p_{I,j}(\mathbf{x}).$$

A component p_I at depth $t = |I|$ has degree at most $d - t$, and each of its degree-ℓ coefficients has magnitude at most $\beta n^{d-t-\ell}$, including when p_I is constant.

Proof. Assigning each nonconstant monomial of p_I to one of its variables, for instance the first in a fixed ordering, gives such a recursive decomposition (Figure 1): a degree-ℓ monomial of p_I becomes a degree-$(\ell - 1)$ monomial of a child at depth $t + 1$ with the same coefficient. Distinct monomials assigned to the same variable leave distinct quotients, so coefficients do not accumulate, and induction on the depth gives the stated bounds. □

Fix an order σ of the variables. We call the decomposition that assigns every nonconstant monomial to its σ-first variable at every level the *σ-decomposition*. It factors each monomial along its variables in the order σ, and Section 3.6 shows how the choice of σ controls the local footprint bound.

3.2 A Quadratic Warm-Up

The error-footprint construction takes its simplest form for quadratic objectives. Let p be an n-variate degree-2, β-smooth polynomial, decomposed by Lemma 3.1 as

$$p(\mathbf{x}) = c + \sum_{i\in[n]} x_i p_i(\mathbf{x}), \qquad p_i(\mathbf{x}) = c_i + \sum_{j\in[n]} x_j p_{i,j},$$

where the second-level coefficients $p_{i,j}$ are constants. Given an oracle prediction $\widehat{\mathbf{x}}$, evaluating each local coefficient at $\widehat{\mathbf{x}}$ gives the linear surrogate $q(\mathbf{x}) = c + \sum_{i\in[n]} x_i p_i(\widehat{\mathbf{x}})$.

How much can prediction errors distort these local coefficients? A misprediction at coordinate j changes p_i by at most $|p_{i,j}|$. If $\mathbf{u} \in [0,1]^n$ is a fractional certificate for the mispredicted coordinates, then p_i can vary within the tolerance $\tau_i(\mathbf{u}) = \sum_j |p_{i,j}| u_j$, and coordinate j carries the footprint weight $F_j = \sum_i |p_{i,j}|$ across all local coefficients. Because the coefficients $p_{i,j}$ are constants, these weights do not depend on $\widehat{\mathbf{x}}$ in the quadratic case. For a budget b, the quadratic error footprint

$$\Phi_b^{(2)} = \max_{\substack{\mathbf{u}\in[0,1]^n \\ \sum_j u_j \le b}} \sum_{j\in[n]} F_j u_j$$

bounds the total change that up to b prediction errors can cause in the local coefficients. The footprint relaxation maximizes the surrogate while keeping every local coefficient within its tolerance:

$$\begin{aligned} \max_{\mathbf{y},\mathbf{u}} \quad & c + \sum_{i\in[n]} y_i p_i(\widehat{\mathbf{x}}) \\ \text{s.t.} \quad & |p_i(\mathbf{y}) - p_i(\widehat{\mathbf{x}})| \le \tau_i(\mathbf{u}) \quad \forall i \in [n], \\ & \sum_{j\in[n]} u_j \le b, \qquad \mathbf{y},\mathbf{u} \in [0,1]^n. \end{aligned} \tag{2-LP}$$

The relaxation links the surrogate to the true objective in two steps. First, every feasible $(\mathbf{y},\mathbf{u})$ has linearization error at most the footprint:

$$|p(\mathbf{y}) - q(\mathbf{y})| \le \sum_{i\in[n]} y_i |p_i(\mathbf{y}) - p_i(\widehat{\mathbf{x}})| \le \sum_{i\in[n]} \tau_i(\mathbf{u}) = \sum_{j\in[n]} F_j u_j \le \Phi_b^{(2)}.$$

Second, if $\widehat{\mathbf{x}}$ differs from an optimal assignment $\mathbf{x}^*$ on a set S of ε coordinates, then $(\mathbf{x}^*, \mathbf{1}_S)$ is feasible at budget ε, so an optimal LP solution $\mathbf{y}$ at this budget satisfies

$$p(\mathbf{y}) \ge q(\mathbf{y}) - \Phi_\varepsilon^{(2)} \ge q(\mathbf{x}^*) - \Phi_\varepsilon^{(2)} \ge p(\mathbf{x}^*) - 2\Phi_\varepsilon^{(2)}.$$

3.3 Footprint Relaxation and LEAP

For general degree d, we apply the same bookkeeping to a fixed recursive decomposition tree of p as in Lemma 3.1. We write $\mathcal{T}$ for the set of generated nodes, use $\emptyset$ for the root, and constrain the non-root nodes whose components have positive degree,

$$\mathcal{I} = \{I \in \mathcal{T} : I \neq \emptyset,\ \deg(p_I) > 0\}.$$

Unlike the quadratic case, a component p_I may itself be nonlinear, so the relaxation works with its one-step linear surrogate at $\widehat{\mathbf{x}}$,

$$q_I(\mathbf{y}) = c_I + \sum_{j\in[n]} y_j p_{I,j}(\widehat{\mathbf{x}}).$$

For $\mathbf{u} \in [0,1]^n$, the tolerance of a node, the footprint weight of a coordinate, and the error footprint at budget b are

$$\tau_I(\mathbf{u}) = \sum_{j\in[n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j, \qquad F_j(\widehat{\mathbf{x}}) = \sum_{I\in\mathcal{I}} |p_{I,j}(\widehat{\mathbf{x}})|, \qquad \Phi_b(\widehat{\mathbf{x}}) = \max_{\substack{\mathbf{u}\in[0,1]^n \\ \sum_j u_j \le b}} \sum_{j\in[n]} F_j(\widehat{\mathbf{x}}) u_j.$$

The guarantees below evaluate the footprint at the prediction error $b = \varepsilon$.

The footprint relaxation is

$$\begin{aligned} \max_{\mathbf{y},\mathbf{u}} \quad & c_\emptyset + \sum_{j\in[n]} y_j p_j(\widehat{\mathbf{x}}) \\ \text{s.t.} \quad & |q_I(\mathbf{y}) - p_I(\widehat{\mathbf{x}})| \le \tau_I(\mathbf{u}) \quad \forall I \in \mathcal{I}, \\ & \sum_{j\in[n]} u_j \le b, \\ & \mathbf{y} \in [0,1]^n, \quad \mathbf{u} \in [0,1]^n. \end{aligned} \tag{d-LP}$$

For $d = 2$, every q_I coincides with p_I, and (d-LP) reduces to (2-LP). Different valid decompositions yield different footprint weights, while the guarantee below applies to any fixed decomposition satisfying Lemma 3.1.

Because p is multilinear, a fractional solution of the footprint relaxation can be rounded deterministically without decreasing the objective value.

Algorithm 1: LEAP: LEarning-augmented Approximation via linear Programming

Input : Problem instance $\pi \in \Pi$, oracle f
Output : Integer solution $\mathbf{z}$
Obtain oracle prediction $\widehat{\mathbf{x}} \leftarrow f(\pi)$
Let p be the objective polynomial of π, replace powers x_i^e with x_i for $e \geq 1$, and collect like monomials
Fix a recursive decomposition tree $\mathcal{T}$ for the resulting multilinear polynomial p, such as the σ-decomposition for a variable order σ
for $b \in \{0, 1, \ldots, n\}$ **do**
  Construct (d-LP) at budget b, and solve it to obtain a fractional solution $(\mathbf{y}_b, \mathbf{u}_b)$
  Obtain $\mathbf{z}_b$ using deterministic rounding
  Yield $\mathbf{z}_b$ and $p(\mathbf{z}_b)$
return *the candidate* $\mathbf{z}_b$ *with the largest evaluated objective value*

Lemma 3.2 (Deterministic rounding for multilinear objectives)**.** *Let $p(\mathbf{x})$ be a multilinear polynomial. For any fractional solution $\mathbf{y} \in [0,1]^n$, deterministic rounding yields an integral vector $\mathbf{z} \in \{0,1\}^n$ such that*

$$p(\mathbf{z}) \geq p(\mathbf{y}).$$

Proof. Round the variables one at a time. When all variables except x_i are fixed, multilinearity makes p affine in x_i. An affine function on $[0,1]$ is maximized at an endpoint, so choosing the better value in $\{0,1\}$ does not decrease the objective. Repeating this for all coordinates gives $p(\mathbf{z}) \geq p(\mathbf{y})$. □

A complete LEAP protocol is given in Algorithm 1. Its rounding step is the deterministic rounding of Lemma 3.2. Since the prediction error is unknown, the algorithm scans all budgets in $\{0, \ldots, n\}$ and returns the best rounded candidate.

3.4 Error-Footprint Guarantee

We analyze the approximation quality of LEAP by first evaluating a fixed error budget b and then accounting for budget selection. To understand the loss of the LP solution, and hence of its rounded assignment, we decompose it into two terms: the distortion caused by linearizing the objective, which can enlarge the gap, and the advantage of the LP solution on the surrogate objective, which reduces it.

Formally, write $q = q_\emptyset$ for the surrogate objective of (d-LP) and $\zeta = p - q$ for its linearization error, which can take either sign. For a budget $b \in [0, n]$, let $P_b \subseteq [0,1]^n$ denote the projection of the feasible region of (d-LP) at budget b onto the $\mathbf{y}$-coordinates. Let $\mathbf{y}_b$ be the optimal solution returned by the LP solver at budget b, with surrogate value $Q_b = q(\mathbf{y}_b) = \max_{\mathbf{x} \in P_b} q(\mathbf{x})$, and let $\mathbf{z}_b$ be its deterministic rounding. For any feasible point $\mathbf{x} \in P_b$, we define the *surrogate gain* of $\mathbf{y}_b$ over $\mathbf{x}$ as $g_b(\mathbf{x}) = Q_b - q(\mathbf{x}) \geq 0$. Whenever an optimal assignment $\mathbf{x}^*$ lies in P_b, the identity $p = q + \zeta$ yields the error decomposition

$$\mathrm{OPT} - p(\mathbf{y}_b) = \underbrace{\left[\zeta(\mathbf{x}^*) - \zeta(\mathbf{y}_b)\right]}_{\text{linearization discrepancy}} - \underbrace{g_b(\mathbf{x}^*)}_{\text{surrogate gain}} . \tag{1}$$

The first term is the net change in linearization error, and the second is the margin achieved by the LP on the surrogate. We first bound the first term using only $g_b(\mathbf{x}^*) \geq 0$, and Section 3.5 then uses LP duality to recover part of the second.

Lemma 3.3 (Footprint control)**.** *For every budget b and every feasible $(\mathbf{y}, \mathbf{u})$ of* (d-LP) *at budget b,*

$$|\zeta(\mathbf{y})| \le \sum_{j \in [n]} F_j(\widehat{\mathbf{x}}) u_j \le \Phi_b(\widehat{\mathbf{x}}).$$

Moreover, if $\mathbf{x}^* \in \{0,1\}^n$ and $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$, then $\mathbf{x}^* \in P_\varepsilon$ and $|\zeta(\mathbf{x}^*)| \le \Phi_\varepsilon(\widehat{\mathbf{x}})$; in particular, $\mathbf{x}^* \in P_b$ for every $b \ge \varepsilon$.

The proof, given in Section C.1, proceeds by induction over the decomposition tree: the linearization error of each component is at most the total tolerance of its constrained descendants, and these tolerances sum to $\sum_j F_j(\widehat{\mathbf{x}}) u_j$ at the root. For the second claim, the indicator vector of the mispredicted coordinates certifies that $\mathbf{x}^*$ is feasible at budget ε.

The surrogate gain is nonnegative, so dropping it from (1) already gives the main guarantee.

Theorem 3.1 (Error-footprint guarantee)**.** *Let p be a multilinear objective, let $\mathbf{x}^*$ be an optimal assignment, and let $\widehat{\mathbf{x}}$ be the oracle prediction. Set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$. For every budget b with $\mathbf{x}^* \in P_b$,*

$$OPT - p(\mathbf{z}_b) \le \Phi_\varepsilon(\widehat{\mathbf{x}}) + \Phi_b(\widehat{\mathbf{x}}).$$

Consequently, Leap *with deterministic rounding returns an integral assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Proof. By Lemma 3.2, (1), and $g_b(\mathbf{x}^*) \ge 0$,

$$\mathrm{OPT} - p(\mathbf{z}_b) \le \mathrm{OPT} - p(\mathbf{y}_b) \le \zeta(\mathbf{x}^*) - \zeta(\mathbf{y}_b).$$

The bounds $\zeta(\mathbf{x}^*) \le \Phi_\varepsilon(\widehat{\mathbf{x}})$ and $-\zeta(\mathbf{y}_b) \le \Phi_b(\widehat{\mathbf{x}})$ of Lemma 3.3 give the first claim. By the same lemma, $\mathbf{x}^* \in P_\varepsilon$, so $\mathrm{OPT} - p(\mathbf{z}_\varepsilon) \le 2\Phi_\varepsilon(\widehat{\mathbf{x}})$, and the full scan returns a candidate at least as good as $\mathbf{z}_\varepsilon$. □

Enumerating the error budget. The proof of Theorem 3.1 uses only one budget that covers the true error, and a budget within a factor of two of the error suffices. The full scan in Algorithm 1 can therefore be replaced by the doubling grid

$$\mathcal{B} = \{0, n\} \cup \{2^j : j \in \mathbb{Z}_{\ge 0},\ 2^j < n\},$$

which has $O(\log n)$ elements.

Proposition 3.1 (Doubling budget grid)**.** *Under the conditions of Theorem 3.1,* Leap *run over the budgets in $\mathcal{B}$ solves $O(\log n)$ linear programs and returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 3\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

The proof, given in Section C.1, uses a grid budget b with $\varepsilon \le b < 2\varepsilon$, for which $\Phi_b(\widehat{\mathbf{x}}) \le 2\Phi_\varepsilon(\widehat{\mathbf{x}})$.

We state the remaining results for the full scan; each extends to the doubling grid with $3\Phi_\varepsilon(\widehat{\mathbf{x}})$ in place of $2\Phi_\varepsilon(\widehat{\mathbf{x}})$, and for Theorem 3.2 with κ_ε^* replaced by κ_b^* at the grid budget b used in the proof of Proposition 3.1.

Complexity. For fixed degree d, the decomposition contributes $O(n^{d-1})$ tuple-indexed constraints and polynomially many variables in the worst case, so each relaxation has polynomial description size. The relaxation is much smaller for sparse objectives. Under the decomposition constructed in the proof of Lemma 3.1, a constrained node lies on the path of some monomial of p, and each monomial passes through at most $d-1$ constrained nodes. The relaxation therefore has at most $(d-1)M_p$ coefficient constraints, where M_p is the number of monomials of p; for MAX-k-SAT with clause set $\mathcal{C}$, $M_p \le 2^k|\mathcal{C}|$. Let N_d denote this description size and $T_{\mathtt{LP}}$ the running time of the chosen polynomial-time LP solver. Since the LP solves dominate the running time, full budget enumeration takes $O(n \cdot T_{\mathtt{LP}}(N_d))$ time, and the doubling grid of Proposition 3.1 reduces this to $O(\log n \cdot T_{\mathtt{LP}}(N_d))$. With a perfect prediction, the error footprint is zero, and for multilinear objectives deterministic rounding returns an optimal solution.

3.5 Refinement via LP Duality

The surrogate gain $g_b(\mathbf{x}^*)$ in (1) is positive whenever $\mathbf{x}^*$ does not maximize the surrogate over P_b. We characterize this gain through LP dual prices and use it to tighten Theorem 3.1.

Write (d-LP) at budget b as

$$\max_{\mathbf{w}} \{c_\emptyset + \bar{\mathbf{a}}^\top \mathbf{w} : \ G\mathbf{w} \le \mathbf{h}\}, \qquad \mathbf{w} = (\mathbf{y}, \mathbf{u}),$$

where the rows of $G\mathbf{w} \le \mathbf{h}$ are the coefficient constraints, the budget constraint, and the bounds on $\mathbf{y}$ and $\mathbf{u}$, and $\bar{\mathbf{a}}$ has zero entries on $\mathbf{u}$. For a feasible $\mathbf{w}$, let $\boldsymbol{\sigma}(\mathbf{w}) = \mathbf{h} - G\mathbf{w} \ge 0$ be its vector of constraint slacks.

Lemma 3.4 (LP gain)**.** *Let $\boldsymbol{\lambda} \ge 0$ be an optimal dual solution of* (d-LP) *at budget b. Every feasible $\mathbf{w} = (\mathbf{x}, \mathbf{u})$ satisfies*

$$g_b(\mathbf{x}) = \boldsymbol{\lambda}^\top \boldsymbol{\sigma}(\mathbf{w}).$$

Proof. Dual feasibility gives $G^\top \boldsymbol{\lambda} = \bar{\mathbf{a}}$, and strong duality gives $Q_b = c_\emptyset + \mathbf{h}^\top \boldsymbol{\lambda}$. Hence $g_b(\mathbf{x}) = Q_b - c_\emptyset - \bar{\mathbf{a}}^\top \mathbf{w} = \boldsymbol{\lambda}^\top (\mathbf{h} - G\mathbf{w})$. □

Intuitively, Lemma 3.4 shows that the surrogate gain prices each constraint slack by its dual multiplier. To quantify what fraction of the linearization discrepancy $\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b)$ is offset by this gain, we define

$$\kappa_b^* = \max\Big\{\kappa \in [0,1] : \ g_b(\mathbf{x}) \ge \kappa\big(\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b)\big) \text{ for every } \mathbf{x} \in P_b\Big\}. \tag{2}$$

The maximum exists, since each $\mathbf{x}$ imposes a closed condition and $\kappa = 0$ always qualifies. By Lemma 3.4, κ_b^* measures how much of the change in linearization error the dual prices of the slacks can pay for: $\kappa_b^* = 0$ recovers the footprint bound of Theorem 3.1, and $\kappa_b^* = 1$ means that $\mathbf{y}_b$ maximizes p over P_b.

Theorem 3.2 (LP-refined guarantee)**.** *Under the conditions of Theorem 3.1, for every budget b with $\mathbf{x}^* \in P_b$,*

$$OPT - p(\mathbf{z}_b) \le (1 - \kappa_b^*)\big[\Phi_\varepsilon(\widehat{\mathbf{x}}) - \zeta(\mathbf{y}_b)\big] \le (1 - \kappa_b^*)\big[\Phi_\varepsilon(\widehat{\mathbf{x}}) + \Phi_b(\widehat{\mathbf{x}})\big].$$

Consequently, Leap *with deterministic rounding returns an integral assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2(1 - \kappa_\varepsilon^*)\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

Proof. By Lemma 3.2, (1), and (2) at $\mathbf{x} = \mathbf{x}^*$,

$$\mathrm{OPT} - p(\mathbf{z}_b) \le \mathrm{OPT} - p(\mathbf{y}_b) = \big[\zeta(\mathbf{x}^*) - \zeta(\mathbf{y}_b)\big] - g_b(\mathbf{x}^*) \le (1 - \kappa_b^*)\big[\zeta(\mathbf{x}^*) - \zeta(\mathbf{y}_b)\big].$$

Since $1 - \kappa_b^* \ge 0$, the bounds $\zeta(\mathbf{x}^*) \le \Phi_\varepsilon(\widehat{\mathbf{x}})$ and $-\zeta(\mathbf{y}_b) \le \Phi_b(\widehat{\mathbf{x}})$ of Lemma 3.3 give the two inequalities. By the same lemma, $\mathbf{x}^* \in P_\varepsilon$, so $\mathrm{OPT} - p(\mathbf{z}_\varepsilon) \le 2(1 - \kappa_\varepsilon^*)\Phi_\varepsilon(\widehat{\mathbf{x}})$, and the full scan returns a candidate at least as good as $\mathbf{z}_\varepsilon$. □

Since $\kappa_b^* \ge 0$, Theorem 3.2 is never weaker than Theorem 3.1, and if $\kappa_b^* = 1$ at a budget with $\mathbf{x}^* \in P_b$, then $\mathbf{z}_b$ is optimal. Section C.2 characterizes when κ_b^* is positive and shows how lower bounds on κ_b^* can be certified from an optimal dual solution.

3.6 Analyzing the Error Footprint

Theorem 3.2 sharpens the guarantee once the LP is solved, while Theorem 3.1 gives an *a priori* guarantee as soon as the error footprint is bounded. We bound $\Phi_\varepsilon(\widehat{\mathbf{x}})$ first globally, using only smoothness (Definition 3.1), and then locally, through the order of the σ-decomposition.

Lemma 3.5 (Global footprint bound)**.** *Let $p(\mathbf{x})$ be an n-variate degree-d, β-smooth polynomial with $d \ge 2$ and $n > d$. There is a constant ρ_d depending only on d such that*

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) \le \rho_d \beta n^{d-1} \varepsilon.$$

The proof, given in Section C.3, bounds the contribution of each level of the decomposition tree by the magnitude bound of Arora et al. (1999, Lemma 3.2).

Corollary 3.1 (Global footprint consequence)**.** *Let p be a multilinear degree-d, β-smooth objective with $d \ge 2$ and $n > d$, let $\mathbf{x}^*$ be an optimal assignment, and set $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$. Under deterministic rounding,* Leap *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \ge OPT - 2\rho_d \beta n^{d-1} \varepsilon.$$

Proof. Combine Theorem 3.1 with Lemma 3.5. □

The global bound ignores which variables appear in each component. To capture local structure, write the multilinear objective as $p(\mathbf{x}) = \sum_{M \subseteq [n]} a_M \mathbf{x}_M$ with $\mathbf{x}_M = \prod_{i \in M} x_i$, fix an order σ of the variables, and build (d-LP) from the σ-decomposition. The *ordered load* of coordinate j is

$$L_j^\sigma = \sum_{M \ni j:\ j \ne \min_\sigma M} |a_M|,$$

the total weight of the monomials that contain j but do not start with j in the order σ. Let $L_{(1)}^\sigma \ge L_{(2)}^\sigma \ge \cdots$ denote the ordered loads in decreasing order.

Lemma 3.6 (Ordered-load footprint bound)**.** *Under the σ-decomposition, every prediction $\widehat{\mathbf{x}} \in \{0,1\}^n$ satisfies $F_j(\widehat{\mathbf{x}}) \le L_j^\sigma$ for every $j \in [n]$, and hence, for every integer budget b,*

$$\Phi_b(\widehat{\mathbf{x}}) \le \sum_{t=1}^{b} L_{(t)}^\sigma \le b \max_{j \in [n]} L_j^\sigma.$$

The proof, given in Section C.3, observes that each monomial charges every variable other than its σ-first one exactly once, with magnitude at most $|a_M|$ because $\widehat{\mathbf{x}}$ is Boolean.

Combined with Theorem 3.1, the lemma bounds the loss of LEAP by $2\sum_{t=1}^{\varepsilon} L^{\sigma}_{(t)}$. The order σ also decides which errors are charged at all, since a mispredicted variable that comes first in every monomial containing it moves no local coefficient; this gives the following sharper bound.

Proposition 3.2 (Structural gain)**.** *Let p be a multilinear objective, let $\mathbf{x}^*$ be an optimal assignment, and build* LEAP *from the σ-decomposition. Let $S = \{j : \widehat{x}_j \neq x^*_j\}$, and let $\varepsilon_\sigma = |\{j \in S : \ L^{\sigma}_j > 0\}| \leq \varepsilon$ count the mispredicted variables with positive ordered load. Then* LEAP *returns an assignment $\mathbf{z}$ satisfying*

$$p(\mathbf{z}) \geq OPT - 2\sum_{t=1}^{\varepsilon_\sigma} L^{\sigma}_{(t)} \geq OPT - 2\varepsilon_\sigma \max_{j\in[n]} L^{\sigma}_j.$$

In particular, if $\varepsilon_\sigma = 0$, then LEAP *returns an optimal assignment.*

The proof, given in Section C.4, shows that $\mathbf{x}^*$ is already feasible at a budget of at most ε_σ; since LEAP scans all budgets, this smaller budget enters only the analysis.

Proposition 3.2 reveals how LEAP exploits the structure of the problem. A prediction error affects the outcome only through the local coefficients it distorts, and the decomposition order decides which coefficients each variable enters. Section 4.1 gives a simple example on MAX-CUT.

4 Applications

We instantiate the framework on two examples. The first is MAX-CUT, a quadratic graph cut problem, and the second is MAX-k-SAT, a Boolean constraint satisfaction problem with higher-order interactions. For each problem, we derive a global bound and a local bound, and the local bound depends on the degeneracy of the instance, which governs the best decomposition order. We then show that the local bounds obtained for both problems have optimal order.

4.1 MAX-CUT

The MAX-CUT objective of Example 1 is quadratic, multilinear, and 2-smooth.

Global bound. Corollary 3.1 gives $p(\mathbf{z}) \geq \mathrm{OPT} - O(n\varepsilon)$, so the approximation ratio is $1 - O(\varepsilon/n^\gamma)$ whenever $\mathrm{OPT} = \Omega(n^{1+\gamma})$ for a constant $\gamma \in [0, 1]$, in particular whenever the average degree is $\Omega(n^\gamma)$.

Local bound. Under the σ-decomposition, L^{σ}_j is twice the number of neighbors of j that precede it in σ. The *degeneracy* $\mathrm{dgn}(G)$ is the smallest r for which some order gives every vertex at most r earlier neighbors, and such an order is computed in linear time (Matula and Beck, 1983). Under this order, $\max_j L^{\sigma}_j \leq 2\,\mathrm{dgn}(G)$, so Proposition 3.2 gives

$$p(\mathbf{z}) \geq \mathrm{OPT} - 4\,\mathrm{dgn}(G)\,\varepsilon_\sigma \geq \mathrm{OPT} - 4\,\mathrm{dgn}(G)\,\varepsilon \geq \mathrm{OPT} - 4\Delta\varepsilon,$$

where Δ is the maximum degree, and the approximation ratio is $1 - O(\mathrm{dgn}(G)\,\varepsilon/n)$ on graphs without isolated vertices. The degeneracy is often much smaller than Δ: it is one on trees, at most five on planar graphs, and at most m on Barabási–Albert graphs in which every new vertex attaches m edges.

By Proposition 3.2, errors at vertices placed before all of their neighbors are not charged at all; the following example consists of such errors only.

Example 3 (Exact recovery on stars)**.** Let G be the disjoint union of ε stars $K_{1,\Delta}$ with $\Delta \geq 2$, and let the oracle predict the all-ones assignment. The prediction errs only at the ε centers and cuts no edge, while $\mathrm{OPT} = \Delta\varepsilon$. Ordering every center before its leaves gives every center zero ordered load, so $\varepsilon_\sigma = 0$ and Leap returns an optimal cut.

4.2 MAX-k-SAT

The MAX-k-SAT objective of Example 2 is multilinear of degree k and 4^k-smooth.

Global bound. Corollary 3.1 gives $p(\mathbf{z}) \geq \mathrm{OPT} - O(n^{k-1}\varepsilon)$, so the approximation ratio is $1 - O(\varepsilon/n^\gamma)$ whenever $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$.

Local bound. For an order σ, let $\mathrm{occ}_\sigma(j)$ count the clauses that contain j but do not start with j in σ, and define the degeneracy of the formula as $\mathrm{dgn}(\phi) = \min_\sigma \max_j \mathrm{occ}_\sigma(j)$. As for graphs, a smallest-last rule computes an order attaining $\mathrm{dgn}(\phi)$ in polynomial time.

Lemma 4.1. *For every order σ, $L_j^\sigma \leq 2^{k-1}\,\mathrm{occ}_\sigma(j)$ for every j. If each literal occurs in at most Δ clauses, then* $\mathrm{dgn}(\phi) \leq 2\Delta$.

Proof. Each clause expands into monomials with coefficients ± 1, at most 2^{k-1} of which contain a given variable j, and all of them start with j when j comes first in the clause. The second claim holds because j lies in at most 2Δ clauses. □

Under an order attaining $\mathrm{dgn}(\phi)$, Proposition 3.2 gives

$$p(\mathbf{z}) \geq \mathrm{OPT} - 2^k\,\mathrm{dgn}(\phi)\,\varepsilon_\sigma \geq \mathrm{OPT} - 2^k\,\mathrm{dgn}(\phi)\,\varepsilon \geq \mathrm{OPT} - 2^{k+1}\Delta\varepsilon,$$

and since $\mathrm{OPT} \geq n/(2k)$ once variables that appear in no clause are removed, the approximation ratio is $1 - O(\mathrm{dgn}(\phi)\,\varepsilon/n)$.

4.3 Optimality of the Local Bounds

The local bounds above grow linearly in $\mathrm{dgn}\cdot\varepsilon$, and Leap attains them without knowing ε. Unless P = NP, no polynomial-time algorithm can guarantee a smaller order, already for MAX-CUT and even when the error ε with respect to a nearest optimal assignment is given as part of the input.

Proposition 4.1 (Worst-case order of the local bound)**.** *Assume $P \neq NP$, and consider deterministic polynomial-time algorithms that receive a MAX-CUT instance, a prediction $\widehat{\mathbf{x}}$, and its error ε.*

(i) On instances of maximum degree at most 4, no such algorithm guarantees $OPT - p(\mathbf{z}) \leq \psi(\varepsilon)\,\varepsilon$ for any function ψ with $\psi(\varepsilon) \to 0$ as $\varepsilon \to \infty$.

(ii) For $\theta \in \{\mathrm{dgn}(G), \Delta\}$, no such algorithm guarantees $OPT - p(\mathbf{z}) \leq \psi(\theta)\,\theta\varepsilon$ for any function ψ with $\psi(\theta) \to 0$ as $\theta \to \infty$.

Both statements also hold for MAX-k-SAT for every fixed $k \geq 2$, with $\theta \in \{\mathrm{dgn}(\phi), \Delta\}$, where Δ is the maximum number of occurrences of a literal, and with the degree bound in (i) replaced by a constant depending only on k.

The proof in Section C.5 reduces from the APX-hardness of MAX-CUT on graphs of maximum degree three (Alimonti and Kann, 2000).[2] Attaching a private leaf to every vertex fixes the error of the all-zero prediction, and replacing every vertex by t copies raises both the degeneracy and the maximum degree to $\Theta(t)$. Adding correctly predicted disjoint edges also rules out approximation ratios $1 - h(\theta\varepsilon/n)$ with $h(s) = o(s)$, so the ratios $1 - O(\mathrm{dgn} \cdot \varepsilon/n)$ derived above have optimal order.

Together, Sections 3 and 4 answer Q1: given any prediction, LEAP runs in polynomial time with an error-footprint guarantee, and on instances of bounded degeneracy no polynomial-time algorithm improves the order of this guarantee. We next turn to Q2 and ask how to learn a good predictor from sampled optimization instances.

5 Learnability of Oracles

This section studies how to select a predictor from sampled optimization instances. Following the PAC perspective of Gupta and Roughgarden (2017), we view each predictor f as inducing an algorithm $\mathcal{A}_f$: its prediction is fed into the footprint relaxation and the subsequent rounding procedure.

5.1 Oracle-Induced Algorithm Classes

Let Π denote the instance space. Each instance $\pi \in \Pi$ is a smooth Boolean polynomial optimization problem of the form (d-IP); write p_π for its objective. We assume, for simplicity of exposition, that all instances in the family have the same fixed size (Section 5.3 discusses instances of different sizes) and that all objective values lie in a common range $[0, H]$ with $H > 0$. Let $\mathcal{D}$ be an unknown but fixed distribution over Π.

Let $\mathcal{F}$ be a hypothesis class of predictors $f : \Pi \to \{0,1\}^n$, viewed as candidate oracles. For $f \in \mathcal{F}$ and instance π, LEAP uses $\widehat{\mathbf{x}} = f(\pi)$ and returns an integral assignment $\mathbf{z}_f(\pi)$; unless stated otherwise, LEAP refers to Algorithm 1 with the full budget scan. Thus f induces a LEAP algorithm $\mathcal{A}_f$, and we write

$$\mathcal{A}_\mathcal{F} = \{\mathcal{A}_f : f \in \mathcal{F}\}$$

for the induced algorithm class.

Let $\mathbf{x}^*(\pi)$ denote an optimal assignment for instance π. The expected value achieved by predictor f is

$$\mathcal{P}(f) \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[p_\pi(\mathbf{z}_f(\pi))\big],$$

and the expected optimum is

$$\mathcal{P}^* \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[p_\pi(\mathbf{x}^*(\pi))\big].$$

5.2 Empirical Cost Minimization

We select predictors according to the performance of the algorithms they induce. Since objective values lie in $[0, H]$, define the cost of $\mathcal{A}_f$ on π as

$$\texttt{COST}(\mathcal{A}_f, \pi) \triangleq H - p_\pi(\mathbf{z}_f(\pi)).$$

For example, in a MAX-k-SAT family with a fixed number of clauses, H may be taken as the number of clauses, and the cost is the number of clauses not satisfied by the returned assignment.

[2] For randomized algorithms that meet their guarantee with probability at least 2/3, the same reduction applies under the assumption NP $\not\subseteq$ BPP.

The expected risk is

$$R(f) \triangleq \mathbb{E}_{\pi\sim\mathcal{D}}\big[\texttt{COST}(\mathcal{A}_f, \pi)\big],$$

and let

$$f^*_{\texttt{cost}} \in \arg\min_{f\in\mathcal{F}} R(f)$$

be a cost-optimal oracle in the hypothesis class. Since $\texttt{COST}(\mathcal{A}_f, \pi) = H - p_\pi(\mathbf{z}_f(\pi))$, we have $R(f) = H - \mathcal{P}(f)$.

Given an i.i.d. sample $S = \{\pi_1, \ldots, \pi_m\}$ from $\mathcal{D}$, empirical risk minimization (ERM) selects

$$\hat{f}_{\mathrm{ERM}} \in \arg\min_{f\in\mathcal{F}} \frac{1}{m}\sum_{i=1}^{m} \texttt{COST}(\mathcal{A}_f, \pi_i).$$

5.3 Pseudo-Dimension Bound

We bound the complexity of the induced cost class. Since the cost is real-valued, the relevant notion is pseudo-dimension, the standard real-valued analogue of VC dimension (Pollard, 1990; Anthony and Bartlett, 1999). For each coordinate $i \in [n]$, let $\mathcal{F}_i$ be the class of i-th coordinate predictors, and define

$$d_{\mathcal{F}} \triangleq \sum_{i=1}^{n} \mathrm{VCdim}(\mathcal{F}_i).$$

We assume $d_{\mathcal{F}} \geq 1$.

Lemma 5.1. *Let $\mathcal{F}$ be a hypothesis class of predictors $f : \Pi \to \{0,1\}^n$, and let $\mathcal{A}_{\mathcal{F}}$ be the induced class of* Leap *algorithms. The pseudo-dimension of the cost functions*

$$\{\pi \mapsto \texttt{COST}(\mathcal{A}_f, \pi) : f \in \mathcal{F}\}$$

satisfies

$$\mathrm{Pdim}(\mathcal{A}_{\mathcal{F}}) \leq C\, d_{\mathcal{F}} \log(e\, d_{\mathcal{F}})$$

for some absolute constant C.

Proof. Throughout the proof, we fix deterministic tie-breaking rules for the LP solver, the rounding procedure, and the final candidate selection. If the algorithm uses internal randomness, we include the random seed in the input domain. Let Ξ denote the seed space, and let $\mathbf{z}_f(\pi; \xi)$ be the assignment returned on π with seed ξ. Define

$$h_f : \Pi \times \Xi \to [0, H], \qquad h_f(\pi, \xi) = H - p_\pi\big(\mathbf{z}_f(\pi; \xi)\big).$$

It suffices to bound the pseudo-dimension of $\mathcal{H} = \{h_f : f \in \mathcal{F}\}$.

Fix a finite set $S = \{(\pi_1, \xi_1), \ldots, (\pi_m, \xi_m)\} \subseteq \Pi \times \Xi$ and thresholds $r_1, \ldots, r_m \in \mathbb{R}$. For each $f \in \mathcal{F}$, define a labeling vector

$$\ell_f(j) = \mathbf{1}[h_f(\pi_j, \xi_j) > r_j].$$

For fixed (π_j, ξ_j), the final cost is determined by the prediction $f(\pi_j)$, since the instance and seed fix the subsequent relaxation and rounding outcomes. Thus the number of possible labelings $\{\ell_f\}$ is at most the number of distinct prediction matrices

$$P_f \in \{0,1\}^{n\times m}, \qquad [P_f]_{i,j} = f_i(\pi_j).$$

Let $d_i = \mathrm{VCdim}(\mathcal{F}_i)$, and assume $m \geq 1$. If $d_i = 0$, the coordinate class yields at most one distinct vector on the sample. For $1 \leq d_i \leq m$, Sauer's lemma (Sauer, 1972) bounds the number of distinct vectors $(f_i(\pi_1), \ldots, f_i(\pi_m))$ by

$$\sum_{s=0}^{d_i} \binom{m}{s} \leq \left(\frac{em}{d_i}\right)^{d_i} \leq (em)^{d_i}.$$

If $d_i > m$, the trivial bound $2^m \leq (em)^{d_i}$ applies. Thus $(em)^{d_i}$ bounds the number of coordinate vectors in all cases. Even when the coordinates of f are coupled, the set of valid prediction matrices is a subset of the Cartesian product of the coordinate-wise projections. Hence the number of distinct prediction matrices is at most

$$\prod_{i=1}^{n} (em)^{d_i} = (em)^{d_{\mathcal{F}}}.$$

If S is pseudo-shattered by $\mathcal{H}$, then all 2^m binary labelings must be realized, so

$$2^m \leq (em)^{d_{\mathcal{F}}}.$$

This inequality fails for $m > C\, d_{\mathcal{F}} \log(e d_{\mathcal{F}})$ with a sufficiently large absolute constant C, which proves the bound. □

Oracle families. The bound applies to predictor classes with controlled coordinate-wise VC dimension. This includes bounded-size GNN- and Transformer-based predictors for graph and formula instances (Selsam et al., 2019; Gasse et al., 2019; Pan et al., 2025), which have finite VC dimension (Bartlett et al., 2019). Our analysis fixes the instance size n. Parameter-sharing predictors such as GNNs and parameterized local search are also defined across instance sizes, and Section B.2 reports selection experiments that transfer candidates from smaller to larger instances. The computational efficiency of the ERM step is a separate question and remains open at this level of generality (Gupta and Roughgarden, 2017; Bartlett et al., 2022).

5.4 End-to-End Learnability Guarantee

The pseudo-dimension bound gives a standard ERM guarantee for the induced algorithm class. We combine this guarantee with the footprint bound of Theorem 3.1 through the following predictor-class quantity. For an instance π, let $\Phi^{\pi}_{\varepsilon}(\widehat{\mathbf{x}})$ denote the error footprint induced by p_π, $\widehat{\mathbf{x}}$, and ε, and define

$$\Lambda^{\Phi}_{\mathcal{F}} \triangleq 2 \inf_{f \in \mathcal{F}} \mathbb{E}_{\pi \sim \mathcal{D}} \left[\Phi^{\pi}_{\|f(\pi) - \mathbf{x}^*(\pi)\|_1}(f(\pi)) \right].$$

This quantity is the footprint loss of the best predictor in $\mathcal{F}$. The next lemma is where the footprint guarantee enters the learnability analysis.

Lemma 5.2 (Transfer lemma)**.** *For the* Leap *pipeline of Algorithm 1,*

$$\mathcal{P}(f^*_{\mathtt{cost}}) \geq \mathcal{P}^* - \Lambda^{\Phi}_{\mathcal{F}}.$$

Proof. By the definition of $f^*_{\mathtt{cost}}$, for every $f \in \mathcal{F}$,

$$R(f^*_{\mathtt{cost}}) \leq R(f).$$

Since $R(f) = H - \mathcal{P}(f)$, this is equivalent to

$$\mathcal{P}(f^*_{\mathtt{cost}}) \geq \mathcal{P}(f).$$

For every $f \in \mathcal{F}$ and instance π, Theorem 3.1 gives

$$p_\pi(\mathbf{z}_f(\pi)) \geq p_\pi(\mathbf{x}^*(\pi)) - 2\Phi^\pi_{\|f(\pi)-\mathbf{x}^*(\pi)\|_1}(f(\pi)).$$

Taking expectations, combining with $\mathcal{P}(f^*_{\texttt{cost}}) \geq \mathcal{P}(f)$, and taking the infimum over $f \in \mathcal{F}$ yields

$$\mathcal{P}(f^*_{\texttt{cost}}) \geq \mathcal{P}^* - \Lambda^\Phi_{\mathcal{F}}.$$

□

Combining the ERM excess-risk bound with the transfer lemma gives the final end-to-end guarantee.

Theorem 5.1. *Under the setup above, fix $\alpha > 0$ and $\delta \in (0, 1]$. If*

$$m \geq C \left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{F}} \log(e d_{\mathcal{F}}) + \log \frac{1}{\delta}\right),$$

then with probability at least $1-\delta$, any empirical risk minimizer $\hat{f}_{\mathrm{ERM}}$ satisfies $R(\hat{f}_{\mathrm{ERM}}) - R(f^*_{\texttt{cost}}) \leq 2\alpha$, and consequently

$$\mathcal{P}(\hat{f}_{\mathrm{ERM}}) \geq \mathcal{P}^* - \Lambda^\Phi_{\mathcal{F}} - 2\alpha.$$

Proof. By Lemma 5.1, the induced cost class has pseudo-dimension $O(d_{\mathcal{F}} \log(e d_{\mathcal{F}}))$. Applying the standard uniform-convergence bound for ERM (Lemma D.2 in Section D) with this bound gives the excess-risk statement after adjusting the absolute constant C. Since $R(f) = H - \mathcal{P}(f)$, the excess-risk bound is equivalent to

$$\mathcal{P}(\hat{f}_{\mathrm{ERM}}) \geq \mathcal{P}(f^*_{\texttt{cost}}) - 2\alpha.$$

Combining this inequality with Lemma 5.2 gives the expected optimization guarantee. □

The footprint bounds in Sections 3 and 4 turn $\Lambda^\Phi_{\mathcal{F}}$ into concrete bounds involving the best expected prediction error

$$\varepsilon_{\mathcal{F}} \triangleq \inf_{f \in \mathcal{F}} \mathbb{E}_{\pi \sim \mathcal{D}}\big[\|f(\pi) - \mathbf{x}^*(\pi)\|_1\big].$$

The global smoothness bound in Corollary 3.1 gives $\Lambda^\Phi_{\mathcal{F}} = O(\beta n^{d-1} \varepsilon_{\mathcal{F}})$. For MAX-$k$-SAT, if every formula in the support of $\mathcal{D}$ has degeneracy at most r, which holds with $r = 2\Delta$ when each literal occurs in at most Δ clauses, and LEAP uses an order attaining it, then Lemma 4.1 and Proposition 3.2 give $\Lambda^\Phi_{\mathcal{F}} = O(r\, \varepsilon_{\mathcal{F}})$; the order depends only on the instance, so Lemma 5.1 still applies.

LP refinement. Using Theorem 3.2 in place of Theorem 3.1 in the proof of Lemma 5.2 multiplies the error footprint inside the expectation defining $\Lambda^\Phi_{\mathcal{F}}$ by $1 - \kappa^*_{\|f(\pi)-\mathbf{x}^*(\pi)\|_1}$, the coefficient of (2) for instance π under prediction $f(\pi)$ with the tie-breaking rules fixed in the proof of Lemma 5.1; any measurable lower bound on this coefficient may be used instead. The resulting loss term is never larger than $\Lambda^\Phi_{\mathcal{F}}$, and ERM is unchanged, since the coefficients enter only the analysis.

Doubling grid. With the doubling grid of Proposition 3.1, Lemma 5.2 and Theorem 5.1 hold with $\Lambda^\Phi_{\mathcal{F}}$ replaced by $\frac{3}{2}\Lambda^\Phi_{\mathcal{F}}$, so the concrete bounds above keep their order. The pseudo-dimension bound uses only that the cost on a fixed instance is determined by the prediction, and Proposition 3.1 replaces 2Φ by 3Φ in the transfer lemma.

6 Experiments

The experiments cover the two levels of the framework, approximation from a given prediction and oracle selection from sampled instances, and then test LEAP at scale. Section 6.1 controls the prediction error exactly and tests the global and local footprint bounds of Sections 3 and 4. Section 6.2 tests oracle selection by empirical induced cost, as analyzed in Section 5. Section 6.3 runs LEAP on large MAX-SAT application benchmarks.

6.1 Solution Quality from Imperfect Predictions

To control the prediction error exactly, we use *planted* instances, which are generated so that a fixed assignment $\mathbf{x}^*$ cuts every edge for MAX-CUT and satisfies every clause for MAX-SAT, and is therefore optimal. The prediction flips a prescribed fraction of the variables in $\mathbf{x}^*$, so the prediction error equals the number of flipped variables. We compare two uses of this prediction: the direct baseline returns the predicted assignment itself, while LEAP passes the prediction through the relaxation-and-rounding pipeline with the full budget scan of Algorithm 1. Both methods are scored by the relative objective gap $(\mathrm{OPT} - p(\mathbf{z}))/\mathrm{OPT}$ for $n \in \{100, 200, 500, 1000\}$ and flip fractions from 0% to 20%; construction details are given in Section B.1.

Dense instances. This regime tests the global smoothness bound of Section 4: with $\mathrm{OPT} = \Theta(n^d)$, it predicts an approximation ratio of $1 - O(\varepsilon/n)$, where ε/n is the fraction of flipped variables. Figure 2 shows that the gap of the direct prediction grows roughly linearly with the flip fraction, while LEAP recovers the planted optimum at every size and flip fraction.

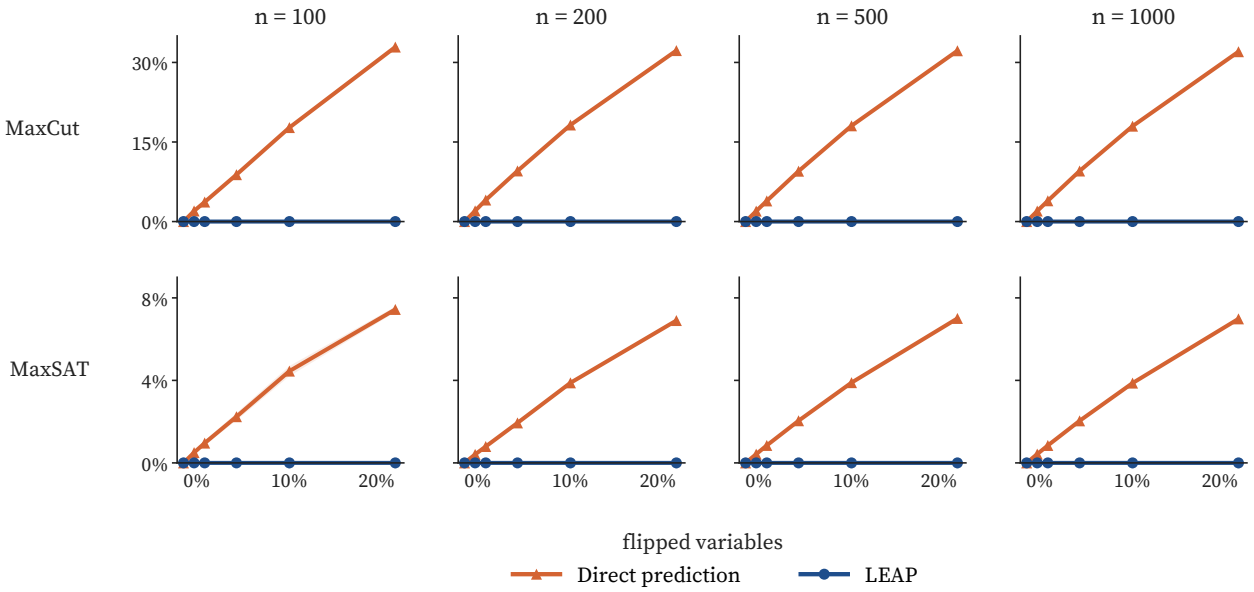


Figure 2: Relative objective gaps versus the fraction of flipped variables for *dense* instances. The gap of the direct prediction grows roughly linearly with the flip fraction, while LEAP repairs the perturbed predictions and recovers the planted optimum at every size.

Sparse instances. This regime tests the local ordered-load bound of Proposition 3.2 on MAX-CUT and bounded-occurrence MAX-SAT instances. Under the natural variable order used here,

each ordered load is at most a constant multiple of the degree or occurrence count, so the bound charges each error only to the edges or clauses it affects. Figure 3 shows that LEAP reduces the gap of the direct prediction at every positive flip fraction, including the largest one; the largest residual gaps occur on the larger MAX-CUT instances.

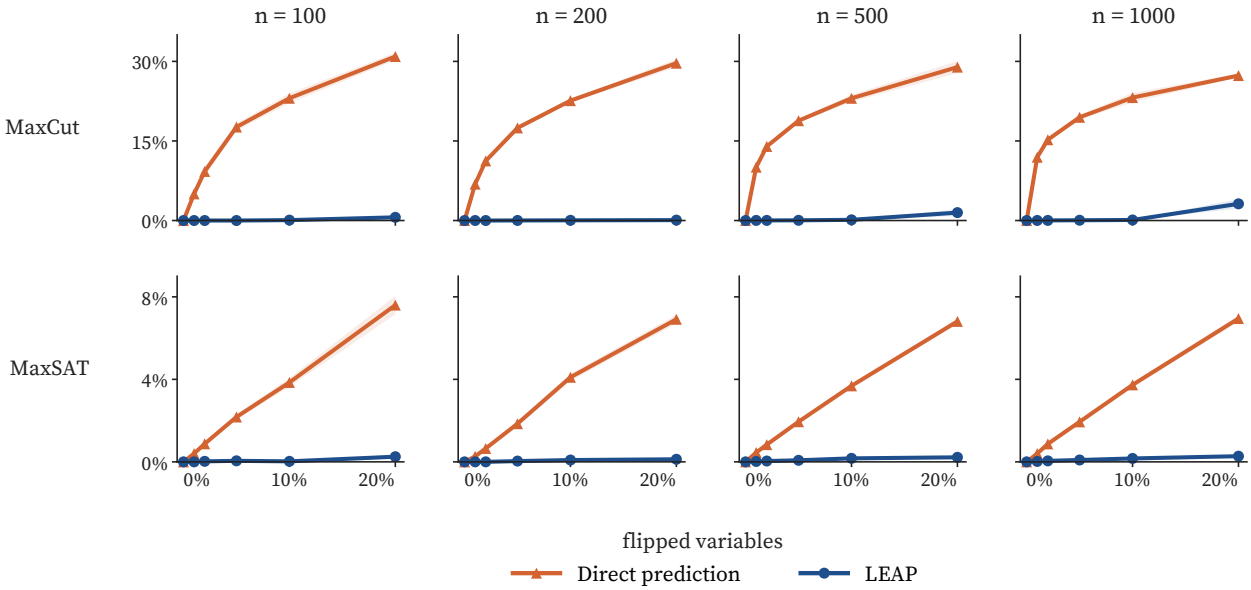


Figure 3: Relative objective gaps versus the fraction of flipped variables for *sparse* instances. LEAP substantially reduces the gap of the direct prediction at every positive flip fraction and instance size; its largest residual gaps occur on the largest MAX-CUT instances.

6.2 Empirical Selection over Oracle Candidates

We next study an empirical version of the learning question in Q2. We consider four finite oracle classes: annealing SLS, WalkSAT-style SLS, ProbSAT-style SLS, and GNN predictors. Within each class, ERM selects the candidate whose induced algorithm has the smallest empirical cost on training instances, and we evaluate the selected candidate on test instances.

We use synthetic MAX-CUT and unweighted MAX-SAT instances with $n = 200$ variables and vary the number m of training instances from 1 to 3000 (see Section B for details). For each oracle class $\mathcal{F}$, Figure 4 plots the excess test risk of the ERM-selected candidate $\hat{f}_m$, that is, its average test cost minus the smallest average test cost over $\mathcal{F}$.

Figure 4 shows that the excess test risk decreases steadily as ERM is given more training instances, consistent with Theorem 5.1.

6.3 Large MAX-SAT Benchmarks

Sections 6.1 and 6.2 evaluate our two theoretical guarantees on controlled instances. Finally, we stress-test the relaxation-and-rounding pipeline of LEAP at scale on MAX-SAT application benchmarks from eight families, comparing its solution quality with nine state-of-the-art MAX-SAT solver configurations (MaxSAT Evaluation, 2024) on the 256 instances for which all methods return an assignment. Benchmark suites of this kind consist of heterogeneous instances and provide no training distribution from which to learn an oracle. We therefore run LEAP in an *iterative*

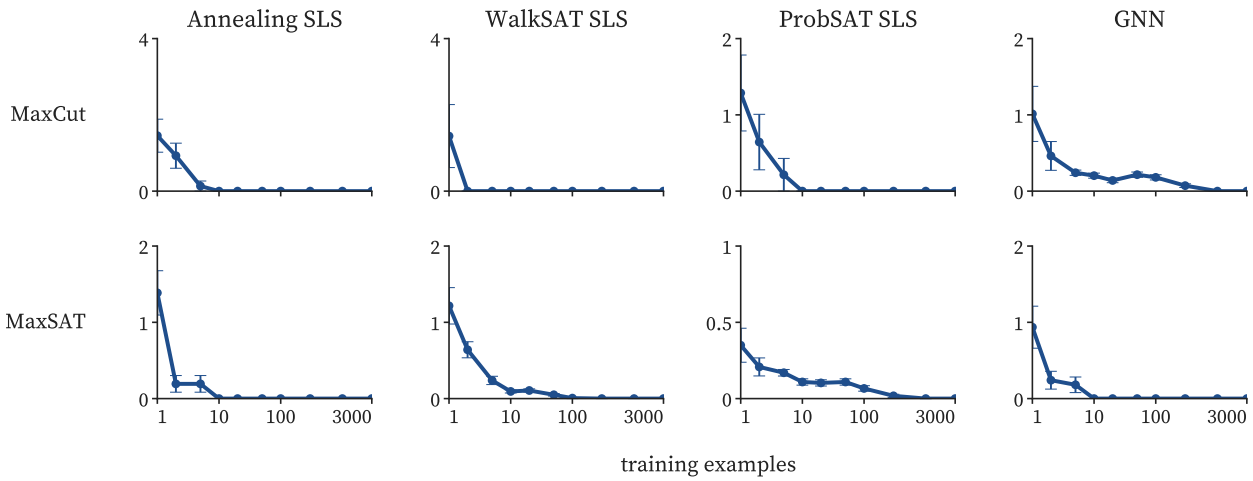


Figure 4: ERM selects useful oracle candidates from training instances. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk, the selected candidate's average test cost minus the smallest average test cost in its oracle class. Error bars show standard errors over repeated draws of the training instances.

self-refinement mode: starting from a uniformly random assignment, Leap runs ten rounds, each using the previous output as its prediction and the doubling grid of Proposition 3.1. Each round solves $O(\log n)$ linear programs, so Leap takes longer than the 60-second baselines; the comparison therefore focuses on solution quality. Table 2 summarizes the sizes of these instances. We report the best of five random-start runs for Leap and one 60-second run per baseline; a single Leap run takes under an hour on most instances, with the longest runs taking tens of hours (see Section B.3 for details). The clause-normalized gap is the number of clauses left unsatisfied by Leap minus that of the best observed baseline, as a percentage of the M clauses.

Family	**Instances**	**Variables**	**Clauses**	**Width**
Deezer	40	25 / 156	169 / 1 684	2.00 / 2
Decision trees	14	27 015 / 82 071	231 206 / 660 735	5.27 / 98
BDD optimization	23	6 491 / 21 190	453 237 / 1 887 499	2.75 / 128
Rule-based classifiers	38	27 611 / 141 265	359 859 / 1 970 958	2.81 / 97
Single-machine scheduling	31	26 343 / 43 672	734 582 / 1 775 621	2.97 / 3
Judgment aggregation	38	12 894 / 22 994	296 276 / 1 567 566	2.89 / 3
Access control	26	1 877 / 2 050	5 043 / 6 900	2.44 / 23
Knowledge-base inconsistency	46	64 919 / 326 096	156 135 / 648 235	2.61 / 121

Table 2: Instance sizes of the eight benchmark families. Entries are mean / maximum over instances; width counts the literals in a clause.

Figure 5 shows that the mean clause-normalized gap is below one percentage point in every family. Together with Table 2, this shows that Leap scales to instances with up to about two million clauses and, even when started from random assignments, stays close to the best observed baseline.

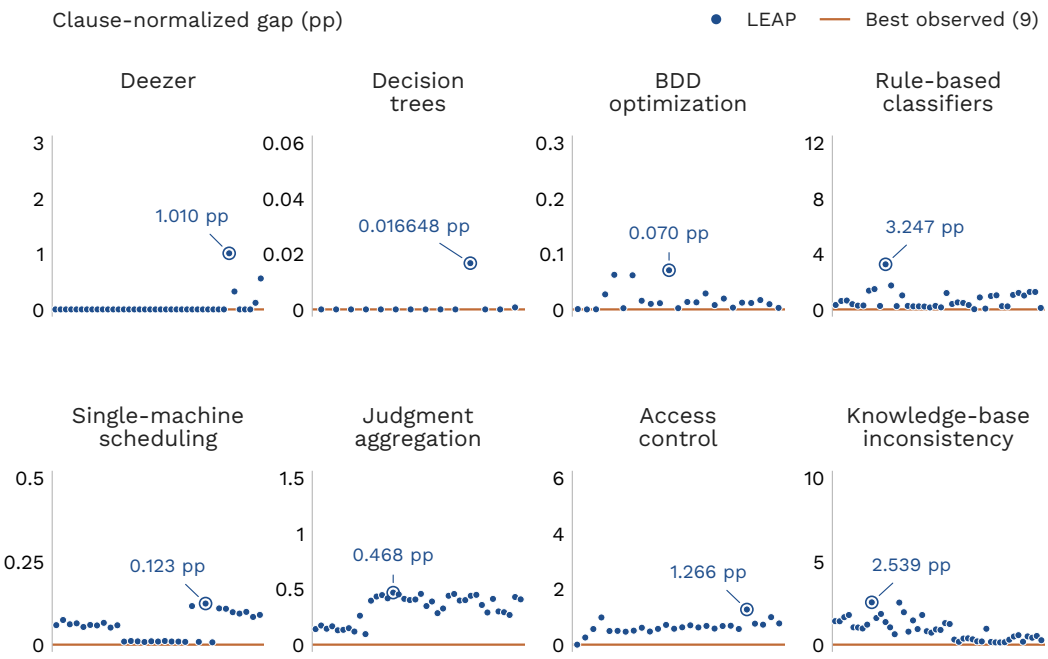


Figure 5: Clause-normalized gaps (percentage points) to the best of nine state-of-the-art solver configurations on each instance. Instances are ordered by size; annotations mark maxima.

7 Concluding Remarks

This work reveals that a learnable prediction model can guide hard smooth integer programs with provable performance guarantees. The loss of Leap is governed by the error footprint of the prediction, that is, by how strongly its errors perturb the local coefficients of the decomposed objective, rather than by the number of errors alone. This sensitivity depends on the structure of the instance and on the decomposition chosen by the algorithm: a suitable variable order limits how far errors propagate, errors that perturb no local coefficient do not enter the guarantee at all, and unless P = NP no polynomial-time algorithm improves the order of the resulting local bound. The experiments are consistent with this picture: Leap substantially repairs imperfect predictions on controlled instances and scales to large MAX-SAT application benchmarks.

This view of prediction error also bears on how oracles should be learned. Since errors at different coordinates have different downstream costs, an oracle is naturally evaluated by the algorithm it induces rather than by coordinate-wise accuracy alone, and our pseudo-dimension bound gives a sample-complexity guarantee for selecting it by empirical induced cost. The main open question lies on the computational side: how to carry out this selection, or the training it calls for, efficiently for expressive predictor classes.

Appendix

This appendix is organized as follows:

- Section A gives extended related-work discussion.
- Section B gives additional experimental details.
- Section C gives omitted approximation-framework proofs, in the order of the main text: footprint control and the doubling grid, the positivity of κ_b^* and its slack certificates, the footprint bounds, the structural gain, and the worst-case lower bound.
- Section D gives omitted proofs for the learnability guarantees.
- Section E verifies the smoothness condition for MAX-k-CSP.

A Extended Related Work

We compare approximation results through their structural assumptions, advice models, and guarantees.

A.1 Detailed Comparison with Approximation Results

Throughout this subsection, $q = \Pr[\text{advice bit correct}] - 1/2$, $\varepsilon = \|\widehat{\mathbf{x}} - \mathbf{x}^*\|_1$, and r denotes the fraction of variables revealed by subset advice.

Classical Approximation and Density. For general MAX-CUT without advice, Goemans and Williamson (1995) achieve approximation ratio $\alpha_{\mathrm{GW}} \approx 0.87856$ by solving an SDP relaxation and applying random-hyperplane rounding. Density assumptions permit stronger guarantees. For dense MAX-CUT and MAX-k-SAT, where the optimum is $\Omega(n^2)$ and $\Omega(n^k)$, respectively, Arora et al. (1999) obtain a $(1-\eta)$-approximation for any fixed $\eta > 0$ through exhaustive sampling, LP relaxation, and rounding. The running time is commonly written as $n^{O(1/\eta^2)}$, with density parameters suppressed in the exponent. Fotakis et al. (2016) extend this approach to almost-sparse MAX-k-CSP instances with $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$ for fixed $\gamma \in (0, 1]$, obtaining a $(1-\eta)$-approximation in $2^{O(n^{1-\gamma}\log n/\eta^3)}$ time.

Sampled Advice. Bampis et al. (2024) incorporate predictions into the sampling framework through a parsimonious one-bit oracle, queried on a sampled set S. For ρ-dense MAX-CUT, the prediction-dependent ratio is $1 - \eta - 8\,\mathrm{err}/(\rho|S|)$, where err counts incorrect predictions in S. Their running time is $O(nT_{\mathrm{LP}})$, where T_{LP} is the time to solve an LP with n variables and $O(n)$ constraints. For general smooth integer programs of degree d and smoothness parameter β, their method requires $O(n\log(\beta nd))$ LP solves.

Label and Edge Advice. Cohen-Addad et al. (2024) obtain an $\alpha_{\mathrm{GW}} + \widetilde{\Omega}(q^4)$ approximation for MAX-CUT from label advice with bias q, with pairwise independence sufficient for their guarantee. Under independent label advice and sufficiently large average degree $\bar{d} = 2|E|/n$, Ghoshal et al. (2025) obtain expected value at least $(1 - O(1/(q\sqrt{\bar{d}})))\mathrm{OPT}$ for unweighted MAX-CUT and Max-2-Lin. With independent q-correlated edge advice, Aamand et al. (2025) achieve $\alpha_{\mathrm{GW}} + \widetilde{\Omega}(q^2)$ for MAX-CUT by aggregating predictions at high-degree vertices and using an approximation subroutine for the remaining part.

Subset Advice. Subset advice reveals the optimal labels of a random fraction r of variables. For MAX-CUT, Cohen-Addad et al. (2024) obtain expected ratio $\alpha_{\mathrm{RT}} + \Omega(r)$, where $\alpha_{\mathrm{RT}} \approx 0.858$ is the max-bisection ratio of Raghavendra and Tan (2012). For MAX-SAT, Attias et al. (2025) show that simplifying the formula with these labels and applying an α-approximation to the remaining instance yields ratio $\alpha + (1 - \alpha)r$.

Relationship to Footprint Analysis. Our framework uses a full assignment prediction and controls its effect through the error footprint $\Phi_\varepsilon(\widehat{\mathbf{x}})$, without requiring independence of prediction errors. For MAX-CUT ($k = 2$) and MAX-k-SAT, global smoothness gives ratio $1 - O(\varepsilon/n^\gamma)$ when $\mathrm{OPT} = \Omega(n^{k-1+\gamma})$, and the local bound gives $1 - O(\Delta\varepsilon/n)$ when each vertex has degree at most Δ or each literal occurs in at most Δ clauses, and more generally $1 - O(\mathrm{dgn} \cdot \varepsilon/n)$ in terms of the degeneracy of the instance. With the doubling grid of Proposition 3.1, LEAP requires $O(\log n)$ LP solves and retains these approximation orders.

A.2 Broader Learning-Augmented Algorithms

Learning-augmented algorithms use predictions to improve performance while studying consistency, degradation with prediction error, and robustness (Lykouris and Vassilvitskii, 2021; Mitzenmacher and Vassilvitskii, 2022). Applications include scheduling (Kumar et al., 2018; Mitzenmacher, 2020; Shahout and Mitzenmacher, 2024), matching (Dinitz et al., 2021), sorting (Bai and Coester, 2023), clustering (Ergun et al., 2022), indexing (Kraska et al., 2018), shortest paths (Lattanzi et al., 2023), paging (Antoniadis et al., 2023), caching (Lykouris and Vassilvitskii, 2018; Im et al., 2022; Chen et al., 2025), and branch-and-bound (Balcan et al., 2018).

B Additional Experimental Details

This appendix provides construction and protocol details for the experiments in Section 6.

B.1 Controlled Perturbation Experiments

For each prediction in either perturbation experiment, we enumerate all integer budgets $b \in \{0, \ldots, n\}$. We build the relaxation from the σ-decomposition with σ the natural variable order. For each budget, we solve the footprint relaxation of (d-LP) and apply deterministic rounding in natural variable order, rounding ties to 1. We return the rounded assignment with the largest original objective value, breaking ties in favor of the smaller budget. For both constructions, the plotted flip fractions are $0\%, 1\%, 2\%, 5\%, 10\%, 20\%$.

Dense constructions. For MAX-CUT, we draw a balanced planted bipartition $\mathbf{x}^*$ and sample $\lceil 0.1n^2 \rceil$ crossing edges, so that $\mathrm{OPT} = |E|$. For MAX-SAT, we draw a planted assignment $\mathbf{x}^*$ and sample $\lceil 0.002n^3 \rceil$ distinct 3-clauses satisfied by $\mathbf{x}^*$, so that $\mathrm{OPT} = |\mathcal{C}|$. We use five independent seeds per problem and size. Within each instance, we fix one random variable ordering and flip progressively longer prefixes of this ordering to create nested perturbations.

Sparse constructions. For MAX-CUT, we sample $2n$ edges across a planted random bipartition, using nonuniform vertex sampling to obtain heterogeneous degrees. For MAX-SAT, we generate $2n$ distinct planted-satisfied 3-clauses, with each variable occurring at most eight times. Thus both constructions have $\mathrm{OPT} = 2n$. For each problem and size in Figure 3, we generate five independent planted instances. Within each instance, we average four perturbations at each flip fraction: two

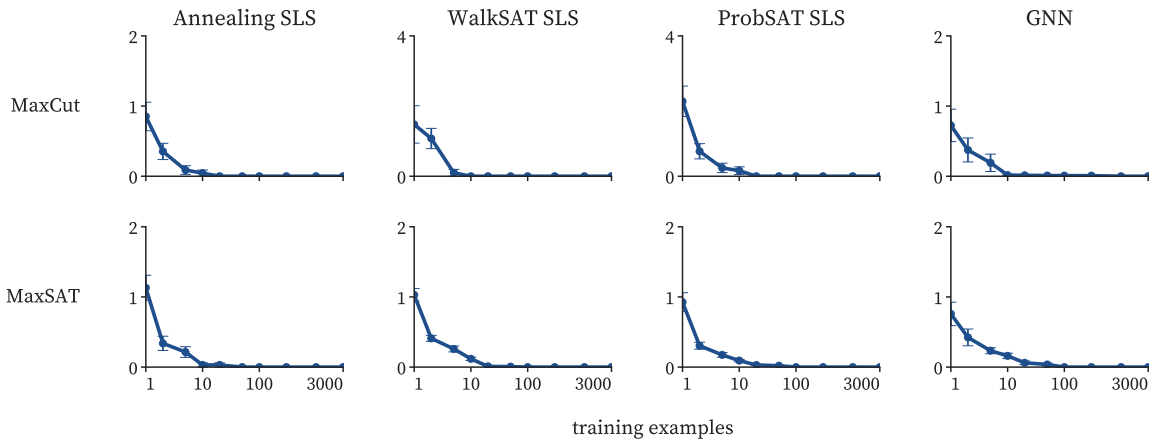


Figure 6: Additional empirical-selection results for $n = 100$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

random flip sets, a highest-load set, and a lowest-load set. Load is graph degree for MAX-CUT and variable occurrence count for MAX-SAT. Curves average the five instance-level means.

B.2 Empirical Selection over Oracle Candidates

For each train-test setting, we use 3000 training instances and 2000 test instances. MAX-CUT instances are Barabási–Albert graphs, and MAX-SAT instances are random 3-CNF formulas with a fixed clause-to-variable ratio. Cost counts uncut edges for MAX-CUT and unsatisfied clauses for MAX-SAT in the assignment returned by Leap. For MAX-CUT, CNF-based candidates use an XOR-CNF encoding, in which each edge (u, v) is represented by clauses enforcing $x_u \neq x_v$; all assignments are scored under the original graph objective.

Table 3 summarizes the four candidate classes. Each local-search variant uses time budgets of $0.0025, 0.005, 0.01, 0.025$ seconds. For each neural architecture, checkpoints after $1, 2, 4$ training epochs define the candidates. The neural training instances are random 3-CNF formulas for MAX-SAT and XOR-CNF encodings of Barabási–Albert graphs for MAX-CUT.

Class	Variants	Count
Annealing SLS	Simulated annealing (Kirkpatrick et al., 1983)	4
WalkSAT-style SLS	GSAT, MaxWalkSAT, weighted and restart MaxWalkSAT (Selman et al., 1992, 1996)	16
ProbSAT-style SLS	ProbSAT-like and SATLike-style search (Balint and Schöning, 2012)	8
GNN predictors	GGNN, NeuroSAT, and GCN (Li et al., 2016; Selsam et al., 2019; Kipf and Welling, 2017)	9

Table 3: Oracle candidates within each class.

For each problem, oracle class, and training size m, we repeat the training draw 50 times and report means and standard errors.

Selection at a Fixed Instance Size. Figures 6 to 8 show results for $n = 100, 500, 1000$. These plots generally show the selected candidates' excess risk declining toward zero as sample counts increase, though some families exhibit intermediate fluctuations.

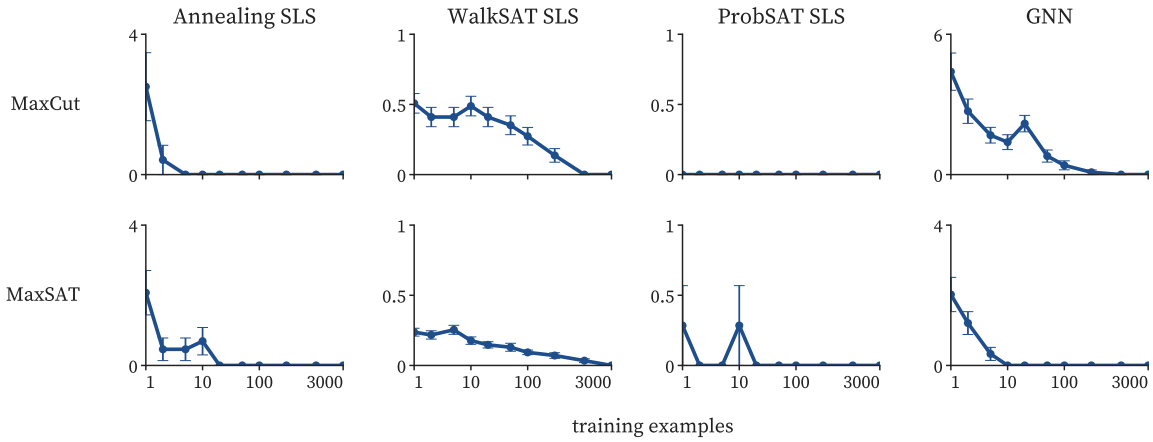


Figure 7: Additional empirical-selection results for $n = 500$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

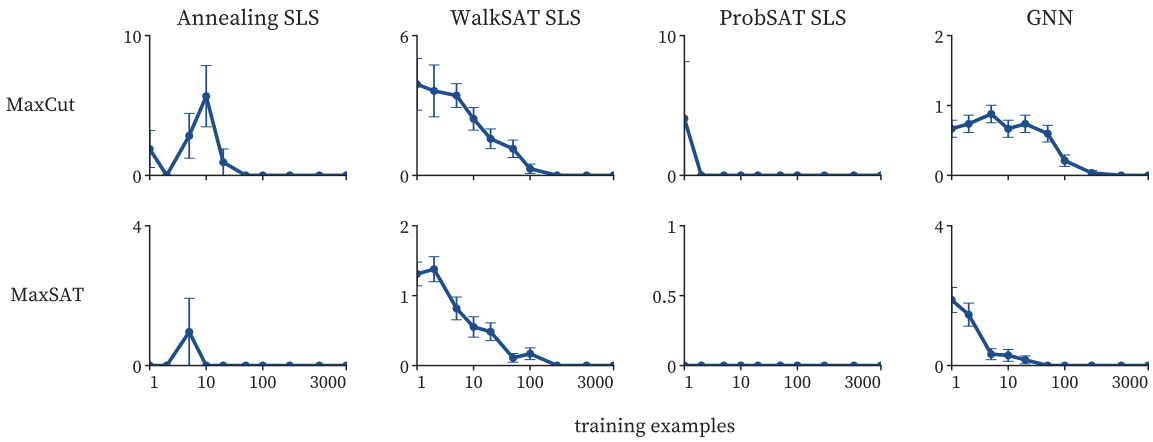


Figure 8: Additional empirical-selection results for $n = 1000$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk.

Selection across Instance Sizes. We evaluate candidates selected on smaller training instances and tested on larger instances. Figures 9 and 10 show results for extrapolation. The selected candidates approach zero excess risk, though a persistent positive gap is observed for the ProbSAT family when transferring from $n = 100$ to $n = 200$ in MAX-SAT.

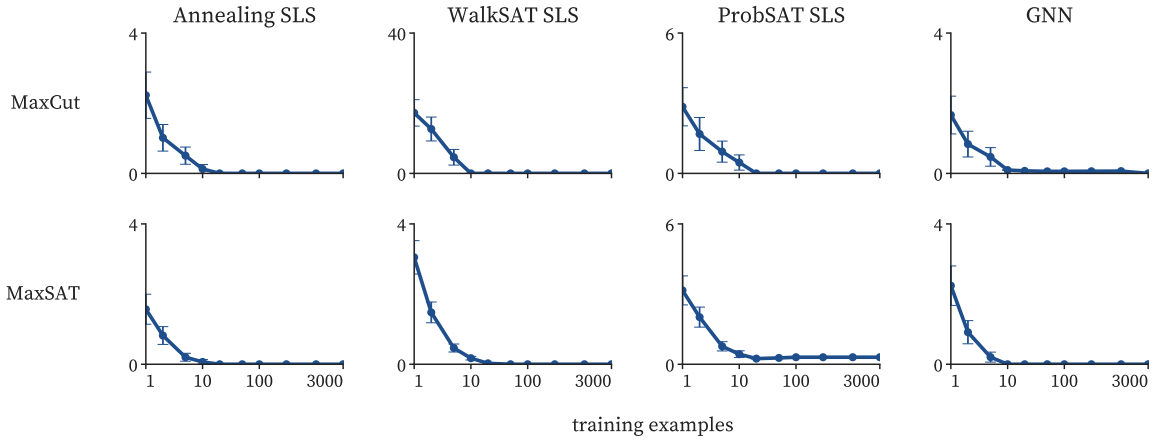


Figure 9: Empirical-selection results for extrapolation from $n = 100$ to $n = 200$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk on the larger test instances.

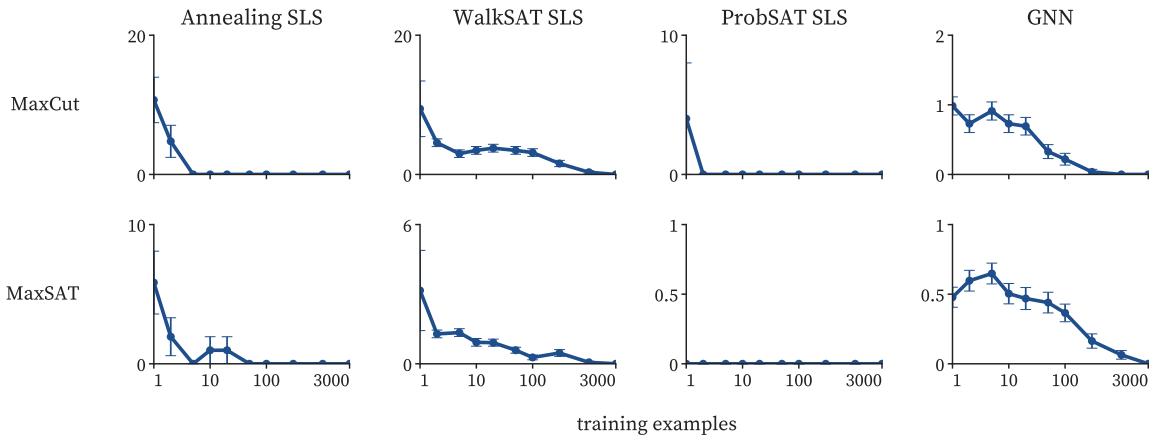


Figure 10: Empirical-selection results for extrapolation from $n = 500$ to $n = 1000$. The horizontal axis shows the number of training instances; the vertical axis shows excess test risk on the larger test instances.

B.3 Details of the Large-Scale MAX-SAT Benchmarks

Seven families are drawn from the historical anytime-unweighted or incomplete-unweighted tracks of the MAX-SAT Evaluation archive (Jabs et al., 2026); the eighth consists of 40 Deezer ego networks from SNAP (Rozemberczki et al., 2020). Table 4 describes their source applications. After deduplication by normalized formula content, we retain 261 instances. The common comparison contains 256 instances; five BDD instances lack assignments from at least one baseline.

All methods use the same formula. For Deezer, the XOR-CNF encoding of each edge makes

Family	Source application
Deezer	Partition social ego networks to maximize crossing edges.
Decision trees	Learn accurate decision trees under depth and size bounds.
BDD optimization	Learn bounded-depth binary decision diagram classifiers.
Rule-based classifiers	Learn Boolean rules balancing classification accuracy and complexity.
Single-machine scheduling	Maximize on-time jobs subject to release times, processing requirements, and dependencies.
Judgment aggregation	Find consistent collective rankings minimizing disagreement with voter or majority preferences.
Access control	Select roles granting required permissions under permission and incompatibility limits.
Knowledge-base inconsistency	Quantify inconsistency through violated statements, contradictory values, or distances to satisfying assignments.

Table 4: Source applications of the eight benchmark families. Their encodings are evaluated as unit-weight all-soft formulas.

the cost the number of uncut edges and gives $M = 2|E|$. Family means average the per-instance clause-normalized gaps.

We use the MSE 2024 versions of Exact, Loandra, NuWLS-c-IBR, SPB-MaxSAT-c, SPB-MaxSAT-c-Band, SPB-MaxSAT-c-FPS, noSAT-MaxSATv3, and the Glucose and IntelSAT variants of tt-open-wbo-inc (MaxSAT Evaluation, 2024). Baselines receive 60 seconds and 12 GiB per instance. LEAP performs ten rounds from each of five random starts, using the output of each round as the prediction for the next, and returns the best final assignment by cost. Each round uses the doubling grid of Proposition 3.1.

C Omitted Material for the Approximation Framework

This section collects the proofs omitted from Sections 3 and 4, in the order in which the results appear.

C.1 Proofs for the Error-Footprint Guarantee

Proof of Lemma 3.3. For a node I, let $\mathcal{T}(I)$ be the set of non-root constrained descendants of I in the decomposition tree. We prove, by induction on the remaining degree of p_I, that every feasible $(\mathbf{y}, \mathbf{u})$ satisfies

$$|p_I(\mathbf{y}) - q_I(\mathbf{y})| \leq \sum_{J \in \mathcal{T}(I)} \tau_J(\mathbf{u}).$$

If p_I is linear, then every nonzero child component is constant, so $p_I(\mathbf{y}) = q_I(\mathbf{y})$.

For the induction step,

$$p_I(\mathbf{y}) - q_I(\mathbf{y}) = \sum_{j \in [n]} y_j (p_{I,j}(\mathbf{y}) - p_{I,j}(\widehat{\mathbf{x}})).$$

Constant child components contribute zero to this difference. For each nonconstant child, we add and subtract its surrogate $q_{I,j}(\mathbf{y})$. The feasibility constraint gives

$$|q_{I,j}(\mathbf{y}) - p_{I,j}(\widehat{\mathbf{x}})| \leq \tau_{I,j}(\mathbf{u}),$$

and the induction hypothesis bounds $|p_{I,j}(\mathbf{y}) - q_{I,j}(\mathbf{y})|$ by the tolerances in the child subtree. Summing over children gives exactly the tolerances over $\mathcal{T}(I)$.

Applying this recurrence at the root, where $\mathcal{T}(\emptyset) = \mathcal{I}$, to $\mathbf{y} \in P_b$ with a certificate $\mathbf{u}$ satisfying $\sum_j u_j \le b$ yields

$$|\zeta(\mathbf{y})| \le \sum_{I \in \mathcal{I}} \tau_I(\mathbf{u}) = \sum_{j \in [n]} F_j(\widehat{\mathbf{x}}) u_j \le \Phi_b(\widehat{\mathbf{x}}).$$

For the second claim, let $S = \{j : \widehat{x}_j \neq x^*_j\}$, so that $|S| = \varepsilon$, and set $u^*_j = \mathbf{1}[j \in S]$. Since $q_I(\widehat{\mathbf{x}}) = p_I(\widehat{\mathbf{x}})$, every $I \in \mathcal{I}$ satisfies

$$|q_I(\mathbf{x}^*) - p_I(\widehat{\mathbf{x}})| = |q_I(\mathbf{x}^*) - q_I(\widehat{\mathbf{x}})| \le \sum_{j \in S} |p_{I,j}(\widehat{\mathbf{x}})| = \tau_I(\mathbf{u}^*).$$

Thus $(\mathbf{x}^*, \mathbf{u}^*)$ is feasible with budget ε, so $\mathbf{x}^* \in P_\varepsilon$, and the first claim gives $|\zeta(\mathbf{x}^*)| \le \Phi_\varepsilon(\widehat{\mathbf{x}})$. Increasing the budget only relaxes the constraint $\sum_j u_j \le b$, so $P_\varepsilon \subseteq P_b$ for every $b \ge \varepsilon$. □

Proof of Proposition 3.1. If $\varepsilon = 0$, the budget $0 \in \mathcal{B}$ gives $p(\mathbf{z}) \ge \mathrm{OPT}$ because $\Phi_0(\widehat{\mathbf{x}}) = 0$. Otherwise, $\mathcal{B}$ contains a budget b with $\varepsilon \le b < 2\varepsilon$, so $\mathbf{x}^* \in P_b$ by Lemma 3.3. Scaling a maximizer of $\Phi_b(\widehat{\mathbf{x}})$ by ε/b shows

$$\Phi_\varepsilon(\widehat{\mathbf{x}}) \le \Phi_b(\widehat{\mathbf{x}}) \le 2\Phi_\varepsilon(\widehat{\mathbf{x}}),$$

and the first bound of Theorem 3.1 gives

$$\mathrm{OPT} - p(\mathbf{z}_b) \le \Phi_\varepsilon(\widehat{\mathbf{x}}) + \Phi_b(\widehat{\mathbf{x}}) \le 3\Phi_\varepsilon(\widehat{\mathbf{x}}).$$

The algorithm returns a candidate at least as good as $\mathbf{z}_b$. □

C.2 Positivity of κ^*_b and Slack Certificates

Whether κ^*_b is positive is decided on the optimal face $\mathcal{O}_b = \arg\max_{\mathbf{x} \in P_b} q(\mathbf{x})$ of the projected LP.

Proposition C.1 (Positivity of κ^*_b)**.** *For every budget b,*

$$\kappa^*_b > 0 \quad \Longleftrightarrow \quad \mathbf{y}_b \in \arg\max_{\mathbf{s} \in \mathcal{O}_b} p(\mathbf{s}).$$

In particular, $\kappa^*_b > 0$ whenever $\mathcal{O}_b$ consists of the single point $\mathbf{y}_b$.

Proof. Since $q = Q_b$ on $\mathcal{O}_b$, the right-hand condition is equivalent to $\zeta(\mathbf{s}) \le \zeta(\mathbf{y}_b)$ for every $\mathbf{s} \in \mathcal{O}_b$. If $\kappa^*_b > 0$, then (2) at $\mathbf{x} = \mathbf{s} \in \mathcal{O}_b$, where $g_b(\mathbf{s}) = 0$, forces $\zeta(\mathbf{s}) \le \zeta(\mathbf{y}_b)$.

Conversely, assume $\zeta \le \zeta(\mathbf{y}_b)$ on $\mathcal{O}_b$. If $\mathcal{O}_b = P_b$, then $\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b) \le 0$ on P_b, and $\kappa^*_b = 1$. Otherwise, fix a norm, let $D > 0$ be the diameter of the polytope P_b, and let $\delta > 0$ be the smallest gap $Q_b - q(\mathbf{v})$ over the vertices $\mathbf{v}$ of P_b outside $\mathcal{O}_b$. Write $\mathbf{x} \in P_b$ as a convex combination of vertices, and let t be its weight on vertices outside $\mathcal{O}_b$. Moving this weight to a vertex of $\mathcal{O}_b$ gives $\mathrm{dist}(\mathbf{x}, \mathcal{O}_b) \le tD$, while $g_b(\mathbf{x}) \ge t\delta$ because q is affine; hence $g_b(\mathbf{x}) \ge \mu \,\mathrm{dist}(\mathbf{x}, \mathcal{O}_b)$ with $\mu = \delta/D$. Since ζ is a polynomial, it is L-Lipschitz on $[0,1]^n$ for some $L \ge 0$. For $\mathbf{x} \in P_b$, let $\mathbf{s} \in \mathcal{O}_b$ be a nearest point; then

$$\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b) \le \zeta(\mathbf{x}) - \zeta(\mathbf{s}) \le L \,\mathrm{dist}(\mathbf{x}, \mathcal{O}_b) \le \frac{L}{\mu}\, g_b(\mathbf{x}).$$

Hence $\kappa^*_b \ge \min\{1, \mu/L\} > 0$, where $\mu/L = +\infty$ if $L = 0$. □

Lower bounds on κ_b^* can be certified from any dual vector that represents the surrogate gain as in Lemma 3.4.

Lemma C.1 (Slack certificate)**.** *Let $\boldsymbol{\lambda} \geq 0$ satisfy $g_b(\mathbf{x}) = \boldsymbol{\lambda}^\top \boldsymbol{\sigma}(\mathbf{w})$ for every feasible $\mathbf{w} = (\mathbf{x}, \mathbf{u})$ at budget b; by Lemma 3.4, every optimal dual solution does. Suppose that a vector $\mathbf{t} \geq 0$ satisfies*

$$\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b) \leq \mathbf{t}^\top \boldsymbol{\sigma}(\mathbf{w}) \qquad \text{for every feasible } \mathbf{w} = (\mathbf{x}, \mathbf{u}).$$

Then

$$\kappa_b^* \geq \min\Big\{1,\ \min_{r:\, t_r > 0} \frac{\lambda_r}{t_r}\Big\},$$

where the inner minimum is $+\infty$ if $\mathbf{t} = 0$.

Proof. Let κ denote the right-hand side. Then $\boldsymbol{\lambda} \geq \kappa \mathbf{t}$ componentwise, and since slacks are nonnegative, $g_b(\mathbf{x}) = \boldsymbol{\lambda}^\top \boldsymbol{\sigma}(\mathbf{w}) \geq \kappa\, \mathbf{t}^\top \boldsymbol{\sigma}(\mathbf{w}) \geq \kappa(\zeta(\mathbf{x}) - \zeta(\mathbf{y}_b))$. Every $\mathbf{x} \in P_b$ has a feasible lift $\mathbf{w}$, so κ is admissible in (2). □

Rescaling a constraint row by a positive factor divides both its dual price and its weight in $\mathbf{t}$ by that factor, so the certified value does not depend on how the constraints are scaled.

C.3 Proofs of the Footprint Bounds

The proof of Lemma 3.5 uses the following magnitude bound for smooth polynomials.

Lemma C.2 (Lemma 3.2, Arora et al. 1999)**.** *Let $p(\mathbf{x})$ be an n-variate β-smooth polynomial of degree at most d. If $n > d$, then for any $\mathbf{x} \in [0,1]^n$, it holds that $|p(\mathbf{x})| \leq 2\,\beta e\, n^d$.*

Proof of Lemma 3.5. Let $\mathcal{I}_r \subseteq \mathcal{I}$ be the set of decomposition tuples of length r. The root objective is not a component constraint, so the relevant levels are $r = 1, \ldots, d-1$. Fix any $\mathbf{u} \in [0,1]^n$ satisfying $\sum_j u_j \leq \varepsilon$.

For $1 \leq r < d-1$, each child component $p_{I,j}$ with $I \in \mathcal{I}_r$ lies at depth $r+1$, so by Lemma 3.1 it has degree at most $d-r-1$ and is β-smooth with respect to this degree bound. By Lemma C.2, applied with degree bound $d-r-1$,

$$|p_{I,j}(\widehat{\mathbf{x}})| \leq 2e\beta n^{d-r-1}.$$

There are at most n^r tuples in $\mathcal{I}_r$, so the level-r contribution is at most

$$\sum_{I \in \mathcal{I}_r} \sum_{j \in [n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j \leq 2e\beta n^{d-1} \varepsilon.$$

There are $d-2$ such levels. At the final level $r = d-1$, the child components lie at depth d and are constant coefficients with magnitude at most β, and therefore

$$\sum_{I \in \mathcal{I}_{d-1}} \sum_{j \in [n]} |p_{I,j}(\widehat{\mathbf{x}})| u_j \leq \beta n^{d-1} \varepsilon.$$

Summing over all levels gives

$$\sum_{j \in [n]} F_j(\widehat{\mathbf{x}}) u_j \leq \big(2e(d-2) + 1\big) \beta n^{d-1} \varepsilon.$$

Taking the maximum over feasible $\mathbf{u}$ proves the claim for any $\rho_d \geq 2e(d-2) + 1$. □

The ordered-load bound follows by tracking where the σ-decomposition charges each monomial.

Proof of Lemma 3.6. Write a nonconstant monomial as $a_M x_{i_1} \cdots x_{i_s}$ with $i_1, \ldots, i_s$ in the order σ. The σ-decomposition factors out i_1 at the root, which is not constrained. If $s \geq 2$, it then factors out each later variable i_{t+1}, $1 \leq t \leq s-1$, at the node $I = (i_1, \ldots, i_t)$, where the monomial contributes $a_M x_{i_{t+2}} \cdots x_{i_s}$ to the child $p_{I,i_{t+1}}$. This node is constrained, since distinct monomials leave distinct quotients and p_I therefore contains the nonconstant term $a_M x_{i_{t+1}} \cdots x_{i_s}$. Since $\widehat{\mathbf{x}}$ is Boolean, the contribution to $p_{I,i_{t+1}}(\widehat{\mathbf{x}})$ has magnitude at most $|a_M|$. Each monomial M therefore adds at most $|a_M|$ to $F_j(\widehat{\mathbf{x}})$ for every $j \in M \setminus \{\min_\sigma M\}$, and the triangle inequality gives $F_j(\widehat{\mathbf{x}}) \leq L_j^\sigma$.

Finally, the maximization defining $\Phi_b(\widehat{\mathbf{x}})$ is a fractional knapsack problem with unit costs and integral budget b, so its value is the sum of the b largest footprint weights, which is at most $\sum_{t=1}^{b} L_{(t)}^\sigma$. □

C.4 Proof of the Structural Gain

Proof of Proposition 3.2. We view the prediction error through the coordinates that actually move a local coefficient. Let $S^+ = \{j \in S : \ F_j(\widehat{\mathbf{x}}) > 0\}$, and call $\varepsilon^+ = |S^+| \leq \varepsilon$ the *effective error*. For $j \in S \setminus S^+$, $F_j(\widehat{\mathbf{x}}) = 0$ forces $p_{I,j}(\widehat{\mathbf{x}}) = 0$ for every $I \in \mathcal{I}$, so these coordinates move no local coefficient. The argument for the second claim of Lemma 3.3, with S replaced by S^+, therefore shows that $(\mathbf{x}^*, \mathbf{1}_{S^+})$ is feasible at budget ε^+, and the first claim gives $\zeta(\mathbf{x}^*) \leq \sum_{j \in S^+} F_j(\widehat{\mathbf{x}}) \leq \Phi_{\varepsilon^+}(\widehat{\mathbf{x}})$. The proof of Theorem 3.1 at budget ε^+ then yields $\mathrm{OPT} - p(\mathbf{z}) \leq 2\Phi_{\varepsilon^+}(\widehat{\mathbf{x}})$.

Finally, Lemma 3.6 gives $F_j(\widehat{\mathbf{x}}) \leq L_j^\sigma$, so $L_j^\sigma = 0$ implies $j \notin S^+$ and hence $\varepsilon^+ \leq \varepsilon_\sigma$. The same lemma at budget ε^+ gives

$$\mathrm{OPT} - p(\mathbf{z}) \leq 2\Phi_{\varepsilon^+}(\widehat{\mathbf{x}}) \leq 2\sum_{t=1}^{\varepsilon^+} L_{(t)}^\sigma \leq 2\sum_{t=1}^{\varepsilon_\sigma} L_{(t)}^\sigma.$$

□

C.5 Worst-Case Order of the Local Bound

We prove Proposition 4.1 by reduction from MAX-CUT on graphs of maximum degree three, which is APX-complete (Alimonti and Kann, 2000, Theorem 4.2). Let H be such a graph with N nonisolated vertices, so $\mathrm{OPT}(H) \geq |E(H)|/2 \geq N/4$. Unless P = NP, polynomial-time loss $o(N)$ is impossible: together with exhaustive search for bounded N, it would give a PTAS.

Construction. Attach a private leaf ℓ_v to each vertex v of H, obtaining $H^\dagger$. For an integer $t \geq 1$, replace each vertex v by t independent copies V_v, and each edge by all t^2 edges between its endpoint groups. Call the resulting graph G_t, and predict the all-zero assignment.

Lemma C.3 (Properties of G_t)**.** *For every $t \geq 1$:*

(a) Every optimal cut is constant on each V_v and separates V_v from V_{ℓ_v}; hence $\varepsilon = tN$.

(b) A cut with loss L yields, in polynomial time, a cut of H with loss at most L/t^2.

(c) $t \leq \mathrm{dgn}(G_t) \leq \Delta \leq 4t$.

Proof. Let w_v be the fraction of ones in V_v, and let p_H be the multilinear cut polynomial of H. Then

$$\frac{\mathrm{cut}_{G_t}(\mathbf{x})}{t^2} = p_H(\mathbf{w}|_H) + \sum_v (w_v + w_{\ell_v} - 2w_v w_{\ell_v}) \le \mathrm{OPT}(H) + N.$$

Here $p_H \le \mathrm{OPT}(H)$ on $[0,1]^N$, and each leaf term is at most one, with equality only for complementary Boolean values. An optimal cut of H with opposite leaf labels attains equality, giving $\mathrm{OPT}(G_t) = t^2(\mathrm{OPT}(H) + N)$. Equality forces every leaf term to be one, proving (a). For a cut with loss L, the same identity gives $\mathrm{OPT}(H) - p_H(\mathbf{w}|_H) \le L/t^2$; deterministic rounding of $\mathbf{w}|_H$ proves (b). Finally, each copy of v has degree $t \deg_{H^\dagger}(v) \le 4t$, and every original edge induces a $K_{t,t}$ of degeneracy t, proving (c). □

Proof for MAX-CUT. Apply Lemma C.3(b). For part (i), set $t = 1$: a guarantee $\psi(\varepsilon)\varepsilon$ gives loss $\psi(N)N = o(N)$ on H. For part (ii), set $t = N$: then $\varepsilon = N^2$ and $N \le \theta \le 4N$ for $\theta \in \{\mathrm{dgn}(G_N), \Delta\}$, so a guarantee $\psi(\theta)\theta\varepsilon$ gives loss at most $\psi(\theta)\theta\varepsilon/N^2 \le 4\psi(\theta)N = o(N)$ on H. Both constructions have polynomial size, contradicting the hardness above.

Approximation ratios. Add N' disjoint edges to G_t, all predicted optimally, with N' polynomial in N and of larger order than both $|E(G_t)|$ and $\theta\varepsilon$. This preserves ε, Δ, and dgn, while $\mathrm{OPT}, n = \Theta(N')$ and $\theta\varepsilon/n \to 0$. A ratio $1 - h(\theta\varepsilon/n)$ with $h(s) = o(s)$ would give loss $o(\theta\varepsilon)$; restricting to G_t cannot increase loss, so the same reductions apply.

Extension to MAX-k-SAT. Encode each edge $\{u, v\}$ by $(x_u \vee x_v)$ and $(\neg x_u \vee \neg x_v)$, giving objective $|E| + \mathrm{cut}(\mathbf{x})$. For $k \ge 3$, extend each 2-clause C by all 2^{k-2} sign patterns on $k-2$ fresh private variables; exactly $2^{k-2} - 1 + \mathbf{1}[C \text{ is satisfied}]$ clauses per block are satisfied. Thus losses are preserved, no clause is repeated, and predicting the fresh variables as zero leaves the nearest-optimum error unchanged. For every $k \ge 2$, the maximum literal occurrence count and the formula degeneracy satisfy

$$t \le \Delta \le 2^{k-2} \cdot 4t, \qquad t \le \mathrm{dgn}(\phi) \le 2^{k+1} t.$$

For the degeneracy lower bound, fix an edge $\{u, v\}$ of $H^\dagger$. In any order, the last variable in $V_u \cup V_v$ has t earlier neighbors, so at least t clauses containing it start earlier; the upper bound follows from Lemma 4.1. The reductions therefore carry over with constants depending only on k.

D Omitted Material for Learnability Guarantees

We recall pseudo-dimension and derive from uniform convergence the ERM guarantee used in Section 5.

Definition D.1 (Pseudo-dimension, Anthony and Bartlett 1999)**.** Let $\mathcal{H}$ be a set of real-valued functions defined on Π. A finite subset $S = \{\pi_1, \ldots, \pi_m\} \subseteq \Pi$ is *(pseudo-)shattered* by $\mathcal{H}$ if there exist real-valued witnesses $r_1, \ldots, r_m$ such that for each subset $T \subseteq S$, there exists a function $h \in \mathcal{H}$ satisfying $h(\pi_i) > r_i$ if and only if $\pi_i \in T$. The *pseudo-dimension* of $\mathcal{H}$, denoted by $\mathrm{Pdim}(\mathcal{H})$, is the maximum cardinality m for which some subset $S \subseteq \Pi$ of size m is pseudo-shattered by $\mathcal{H}$.

Lemma D.1 (Uniform convergence)**.** *Let $H > 0$, and let $\mathcal{H}$ be a nonempty class of functions mapping Π to $[0, H]$, with finite pseudo-dimension $d_{\mathcal{H}}$. Assume the suprema below are measurable. For any distribution $\mathcal{D}$ over Π, any precision $\alpha > 0$, and any confidence parameter $\delta \in (0, 1]$, if the sample size satisfies*

$$m \geq C \left(\frac{H}{\alpha}\right)^2 \left(d_{\mathcal{H}} + \log\left(\frac{2}{\delta}\right)\right), \tag{3}$$

where C is an absolute constant, then with probability at least $1 - \delta$ over an i.i.d. sample $(\pi_1, \ldots, \pi_m) \sim \mathcal{D}^m$, the following uniform bound holds:

$$\sup_{h \in \mathcal{H}} \left| \mathbb{E}_{\pi \sim \mathcal{D}}[h(\pi)] - \frac{1}{m} \sum_{i=1}^{m} h(\pi_i) \right| \leq \alpha.$$

Proof. Let $\mathcal{G} = \{h/H : h \in \mathcal{H}\}$. Its fat-shattering dimension at every positive scale is at most $d_{\mathcal{H}}$. By Mendelson and Vershynin (2003, Theorem 1), its $L_2(Q)$ covering number satisfies

$$\log N(t, \mathcal{G}, L_2(Q)) \leq C_0 d_{\mathcal{H}} \log(2/t) \qquad (0 < t < 1)$$

for every probability measure Q. For a sample $S = (\pi_1, \ldots, \pi_m)$, let Q_S be its empirical measure and let $\sigma_1, \ldots, \sigma_m$ be independent uniform signs in $\{-1, 1\}$. Dudley's entropy integral bounds the empirical Rademacher complexity as

$$\begin{aligned} \widehat{\mathfrak{R}}_S(\mathcal{G}) &\triangleq \mathbb{E}_\sigma \left[\sup_{g \in \mathcal{G}} \frac{1}{m} \sum_{i=1}^{m} \sigma_i g(\pi_i) \right] \\ &\leq \frac{C_1}{\sqrt{m}} \int_0^1 \sqrt{\log N(t, \mathcal{G}, L_2(Q_S))}\, dt \leq C_2 \sqrt{\frac{d_{\mathcal{H}}}{m}}, \end{aligned}$$

since $\int_0^1 \sqrt{\log(2/t)}\, dt$ is finite. Symmetrization, bounded differences, and a union bound over the two one-sided deviations give a uniform deviation of at most $C_3 H \sqrt{(d_{\mathcal{H}} + \log(2/\delta))/m}$ with probability at least $1 - \delta$ (Bartlett and Mendelson, 2002). All constants are absolute. The sample size in (3) makes this bound at most α. □

The following lemma records the ERM consequence used in the main text.

Lemma D.2. *Fix $H > 0$, $\alpha > 0$, $\delta \in (0, 1]$, instance space Π, and cost function $\texttt{COST} : \mathcal{A} \times \Pi \to [0, H]$. Suppose the algorithm class $\mathcal{A}$ induces a cost function class with pseudo-dimension at most $d_{\mathcal{A}}$. If m satisfies (3) with $d_{\mathcal{H}} = d_{\mathcal{A}}$, then with probability at least $1 - \delta$, any empirical risk minimizer $\hat{A}$ satisfies*

$$\mathbb{E}_{\pi \sim \mathcal{D}}[\texttt{COST}(\hat{A}, \pi)] - \min_{A \in \mathcal{A}} \mathbb{E}_{\pi \sim \mathcal{D}}[\texttt{COST}(A, \pi)] \leq 2\alpha.$$

Proof. Let $\mathcal{H} = \{h_A : \Pi \to [0, H] \mid A \in \mathcal{A}\}$, where $h_A(\pi) := \texttt{COST}(A, \pi)$. By assumption, $\mathrm{Pdim}(\mathcal{H}) \leq d_{\mathcal{A}}$. If m satisfies (3), then Lemma D.1 gives, with probability at least $1 - \delta$,

$$\sup_{A \in \mathcal{A}} \left| \mathbb{E}_{\pi \sim \mathcal{D}}[h_A(\pi)] - \tfrac{1}{m} \sum_{i=1}^{m} h_A(\pi_i) \right| \leq \alpha.$$

Let $A_{\mathcal{D}} \in \arg\min_{A \in \mathcal{A}} \mathbb{E}_{\pi \sim \mathcal{D}}[h_A(\pi)]$ be the optimal algorithm, and let $\hat{A}$ be the ERM solution minimizing $\frac{1}{m}\sum_{i=1}^{m} h_A(\pi_i)$. Then

$$\begin{aligned}\mathbb{E}_{\pi \sim \mathcal{D}}[h_{\hat{A}}(\pi)] &\leq \tfrac{1}{m}\sum_{i=1}^{m} h_{\hat{A}}(\pi_i) + \alpha \\ &\leq \tfrac{1}{m}\sum_{i=1}^{m} h_{A_{\mathcal{D}}}(\pi_i) + \alpha \\ &\leq \mathbb{E}_{\pi \sim \mathcal{D}}[h_{A_{\mathcal{D}}}(\pi)] + 2\alpha,\end{aligned}$$

where the first and third inequalities follow from uniform convergence, and the second from the definition of ERM. □

E Smoothness of MAX-k-CSP

This section verifies the smoothness condition for the standard polynomial formulation of MAX-k-CSP.

Every problem in MAX-SNP L-reduces to MAX-k-CSP for some constant k (Papadimitriou and Yannakakis, 1991; Papadimitriou, 1994). Specific graph problems such as MAX-CUT admit low-degree polynomial formulations, as in Example 1.

We represent MAX-k-CSP as a smooth integer program of degree k by arithmetization. Consider an instance with n decision variables $x_1, \ldots, x_n \in \{0,1\}$ and a constraint set $\mathcal{C}$. Each constraint $c \in \mathcal{C}$ is defined on a k-variable scope and is specified by a Boolean function $c : \{0,1\}^k \to \{0,1\}$.

We encode each constraint as a polynomial. For a constraint c on variables $(x_{i_1}, \ldots, x_{i_k})$, define $t_c(\mathbf{x})$ as the sum of indicator polynomials over its satisfying assignments. For each assignment $a \in \{0,1\}^k$ with $c(a) = 1$, the indicator polynomial is

$$\delta_a(\mathbf{x}) = \prod_{r=1}^{k} x_{i_r}^{a_r}(1 - x_{i_r})^{1-a_r}.$$

Then, $t_c(\mathbf{x}) = \sum_{a : c(a)=1} \delta_a(\mathbf{x})$. The objective is the sum over all constraints:

$$p(\mathbf{x}) = \sum_{c \in \mathcal{C}} t_c(\mathbf{x}).$$

This function is a polynomial of degree at most k.

To bound the smoothness parameter, let μ be the maximum number of constraints defined on any single k-variable scope.

- For MAX-CUT on simple graphs, any pair of vertices has at most one edge, so $\mu = 1$.
- For MAX-k-SAT, multiple clauses may involve the same set of variables, for example $x_1 \vee x_2 \vee x_3$ and $\neg x_1 \vee x_2 \vee x_3$. If duplicate identical clauses are excluded, then $\mu \leq 2^k$.

The following lemma bounds the smoothness parameter in terms of μ and k.

Lemma E.1. *The polynomial objective $p(\mathbf{x})$ derived from a MAX-k-CSP instance is β-smooth for any $\beta \geq \mu 2^k$.*

Proof. We bound each monomial coefficient in the expansion of $p(\mathbf{x})$.

For any single constraint c on a set of k variables, the polynomial $t_c(\mathbf{x})$ is a sum of at most 2^k terms of the form $\delta_a(\mathbf{x})$. The expansion of each $\delta_a(\mathbf{x})$ produces monomials with coefficients ± 1. Hence every monomial coefficient contributed by a single constraint has magnitude at most 2^k.

A monomial on a set S of $\ell \leq k$ variables receives contributions only from constraints whose scope contains S. There are $\binom{n-\ell}{k-\ell} \leq n^{k-\ell}$ such scopes, each carrying at most μ constraints, so the coefficient of this monomial has magnitude at most $\mu 2^k n^{k-\ell} \leq \beta n^{k-\ell}$. □

References

Anders Aamand, Justin Y. Chen, Siddharth Gollapudi, Sandeep Silwal, and Hao Wu. Improved approximations for hard graph problems using predictions. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 73–101, Vancouver, Canada, 2025. PMLR.

Paola Alimonti and Viggo Kann. Some APX-completeness results for cubic graphs. *Theoretical Computer Science*, 237(1–2):123–134, 2000. doi: 10.1016/S0304-3975(98)00158-3.

Martin Anthony and Peter L Bartlett. *Neural network learning: Theoretical foundations*. Cambridge University Press, Cambridge, UK, 1999.

Antonios Antoniadis, Joan Boyar, Marek Elias, Lene Monrad Favrholdt, Ruben Hoeksma, Kim S. Larsen, Adam Polak, and Bertrand Simon. Paging with succinct predictions. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 952–968, Honolulu, Hawaii, USA, 2023. PMLR.

Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998.

Sanjeev Arora, David Karger, and Marek Karpinski. Polynomial time approximation schemes for dense instances of NP-hard problems. *Journal of Computer and System Sciences*, 58(1):193–210, 1999.

Idan Attias, Xing Gao, and Lev Reyzin. Learning-augmented algorithms for Boolean satisfiability. arXiv:2505.06146, 2025.

Xingjian Bai and Christian Coester. Sorting with predictions. In *Advances in Neural Information Processing Systems*, volume 36, New Orleans, Louisiana, USA, 2023. Curran Associates, Inc.

Maria-Florina Balcan, Vaishnavh Nagarajan, Ellen Vitercik, and Colin White. Learning-theoretic foundations of algorithm configuration for combinatorial partitioning problems. In *Proceedings of the 30th Conference on Learning Theory*, volume 65, pages 213–274, Amsterdam, Netherlands, 2017. PMLR.

Maria-Florina Balcan, Travis Dick, Tuomas Sandholm, and Ellen Vitercik. Learning to branch. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 344–353, Stockholm, Sweden, 2018. PMLR.

Maria-Florina Balcan, Siddharth Prasad, Tuomas Sandholm, and Ellen Vitercik. Sample complexity of tree search configuration: Cutting planes and beyond. In *Advances in Neural Information Processing Systems*, volume 34, Virtual, 2021. Curran Associates, Inc.

Maria-Florina Balcan, Dan Deblasio, Travis Dick, Carl Kingsford, Tuomas Sandholm, and Ellen Vitercik. How much data is sufficient to learn high-performing algorithms? *Journal of the ACM*, 71(5):32:1–32:58, 2024.

Adrian Balint and Uwe Schöning. Choosing probability distributions for stochastic local search and the role of make versus break. In *Theory and Applications of Satisfiability Testing – SAT 2012*, volume 7317, pages 16–29, Trento, Italy, 2012. Springer.

Evripidis Bampis, Bruno Escoffier, and Michalis Xefteris. Parsimonious learning-augmented approximations for dense instances of NP-hard problems. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 2700–2714, Vienna, Austria, 2024. PMLR.

Peter Bartlett, Piotr Indyk, and Tal Wagner. Generalization bounds for data-driven numerical linear algebra. In *Proceedings of the 35th Conference on Learning Theory*, volume 178, pages 2013–2040, London, UK, 2022. PMLR.

Peter L. Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3:463–482, 2002.

Peter L Bartlett, Nick Harvey, Christopher Liaw, and Abbas Mehrabian. Nearly-tight VC-dimension and pseudodimension bounds for piecewise linear neural networks. *Journal of Machine Learning Research*, 20(63):1–17, 2019.

Peng Chen, Hailiang Zhao, Jiaji Zhang, Xueyan Tang, Yixuan Wang, and Shuiguang Deng. Robustifying Learning-Augmented caching efficiently without compromising 1-Consistency. In *Advances in Neural Information Processing Systems*, volume 38, pages 136021–136064, San Diego, California, USA, 2025. Curran Associates, Inc.

Vincent Cohen-Addad, Tommaso d'Orsi, Anupam Gupta, Euiwoong Lee, and Debmalya Panigrahi. Learning-augmented approximation algorithms for maximum cut and related problems. In *Advances in Neural Information Processing Systems*, volume 37, pages 25961–25980, Vancouver, Canada, 2024. Curran Associates, Inc.

Michael Dinitz, Sungjin Im, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Faster matchings via learned duals. In *Advances in Neural Information Processing Systems*, volume 34, Virtual, 2021. Curran Associates, Inc.

Jon C. Ergun, Zhili Feng, Sandeep Silwal, David Woodruff, and Samson Zhou. Learning-augmented k-means clustering. In *Proceedings of the 10th International Conference on Learning Representations*, Virtual, 2022.

Dimitris Fotakis, Michael Lampis, and Vangelis Th. Paschos. Sub-exponential approximation schemes for CSPs: From dense to almost sparse. In *33rd Symposium on Theoretical Aspects of Computer Science (STACS 2016)*, volume 47, pages 37:1–37:14, Orléans, France, 2016. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32, Vancouver, Canada, 2019. Curran Associates, Inc.

Suprovat Ghoshal, Konstantin Makarychev, and Yury Makarychev. Constraint satisfaction problems with advice. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1202–1221, New Orleans, Louisiana, USA, 2025. SIAM.

Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

Rishi Gupta and Tim Roughgarden. A PAC approach to application-specific algorithm selection. *SIAM Journal on Computing*, 46(3):992–1017, 2017.

Sungjin Im, Ravi Kumar, Aditya Petety, and Manish Purohit. Parsimonious learning-augmented caching. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pages 9588–9601, Baltimore, Maryland, USA, 2022. PMLR.

Christoph Jabs, Dieter Vandesande, Jeremias Berg, Johannes Fichte, George Katsirelos, Ruben Martins, André Schidler, and Dominik Schreiber. MaxSAT evaluation: Complete benchmark collection. Zenodo, 2026. URL `https://zenodo.org/records/21283148`.

Mikhail Khodak, Maria-Florina Balcan, Ameet Talwalkar, and Sergei Vassilvitskii. Learning predictions for algorithms with predictions. In *Advances in Neural Information Processing Systems*, volume 35, New Orleans, Louisiana, USA, 2022. Curran Associates, Inc.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, Toulon, France, 2017.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data*, pages 489–504, Houston, TX, USA, 2018. ACM.

Ravi Kumar, Manish Purohit, and Zoya Svitkina. Improving online algorithms via ML predictions. In *Advances in Neural Information Processing Systems*, pages 9684–9693, Montréal, Canada, 2018. Curran Associates, Inc.

Silvio Lattanzi, Ola Svensson, and Sergei Vassilvitskii. Speeding up Bellman Ford via minimum violation permutations. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 18584–18598, Honolulu, Hawaii, USA, 2023. PMLR.

Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *Proceedings of the 4th International Conference on Learning Representations*, San Juan, Puerto Rico, 2016.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 3296–3305, Stockholm, Sweden, 2018. PMLR.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *Journal of the ACM*, 68(4):24:1–24:25, 2021.

David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. *Journal of the ACM*, 30(3):417–427, 1983. doi: 10.1145/2402.322385.

MaxSAT Evaluation. MaxSAT evaluation 2024, 2024. URL `https://maxsat-evaluations.github.io/2024/mse24-talk.pdf`.

Shahar Mendelson and Roman Vershynin. Entropy and the combinatorial dimension. *Inventiones Mathematicae*, 152(1):37–55, 2003.

Michael Mitzenmacher. Scheduling with predictions and the price of misprediction. In *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*, volume 151, pages 14:1–14:18, Seattle, Washington, USA, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.

Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022.

Leyan Pan, Vijay Ganesh, Jacob Abernethy, Chris Esposo, and Wenke Lee. Can transformers reason logically? A study in SAT solving. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 47632–47671, Vancouver, Canada, 2025. PMLR.

Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, MA, USA, 1994.

Christos H. Papadimitriou and Mihalis Yannakakis. Optimization, approximation, and complexity classes. *Journal of Computer and System Sciences*, 43(3):425–440, 1991.

David Pollard. *Empirical processes: Theory and applications*. Institute of Mathematical Statistics, Hayward, CA, USA, 1990.

Prasad Raghavendra and Ning Tan. Approximating CSPs with global cardinality constraints using SDP hierarchies. In *Proceedings of the 23rd Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 373–387, Kyoto, Japan, 2012. SIAM.

Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate Club: An API oriented open-source Python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, pages 3125–3132, Virtual Event, Ireland, 2020. ACM.

Norbert Sauer. On the density of families of sets. *Journal of Combinatorial Theory, Series A*, 13(1): 145–147, 1972.

Bart Selman, Hector J. Levesque, and David G. Mitchell. A new method for solving hard satisfiability problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, pages 440–446, San Jose, California, USA, 1992. AAAI Press.

Bart Selman, Henry A. Kautz, and Bram Cohen. Local search strategies for satisfiability testing. In *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge*, pages 521–531. American Mathematical Society, Providence, RI, USA, 1996.

Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *Proceedings of the 7th International Conference on Learning Representations*, New Orleans, Louisiana, USA, 2019.

Rana Shahout and Michael Mitzenmacher. SkipPredict: When to invest in predictions for scheduling. In *Advances in Neural Information Processing Systems*, Vancouver, Canada, 2024. Curran Associates, Inc.